

```
0001 | // ===== App.uvue =====
0002 | <script lang="uts">
0003 | import { state, setLifeCycleNum, checkSystemTheme, setNetless } from '@store/index.uts'
0004 | // #ifdef APP-ANDROID || APP-HARMONY
0005 | let firstBackTime = 0
0006 | // #endif
0007 | export default {
0008 |   globalData: {
0009 |     str: 'default globalData str',
0010 |     num: 0,
0011 |     bool: false,
0012 |     obj: {
0013 |       str: 'default globalData obj str',
0014 |       num: 0,
0015 |       bool: false,
0016 |     } as UTSJSONObject,
0017 |     null: null as string | null,
0018 |     arr: [] as number[],
0019 |     mySet: new Set<string>(),
0020 |     myMap: new Map<string, any>(),
0021 |     func: () : string => {
0022 |       return 'globalData func'
0023 |     },
0024 |     launchOptions: {
0025 |       path: '',
0026 |     } as OnLaunchOptions,
0027 |     onShowOption: {
0028 |       path: ''
0029 |     } as OnShowOptions
0030 |   },
0031 |   onLaunch: function (res : OnLaunchOptions) {
0032 |     this.globalData.launchOptions = res
0033 |     // 自动化测试
0034 |     setLifeCycleNum(state.lifeCycleNum + 1000)
0035 |     console.log('App Launch')
0036 |     // 页面性能分析
0037 |     // const performance = uni.getPerformance()
0038 |     // const observer1: PerformanceObserver = performance.createObserver(
0039 |     //   (entryList: PerformanceObserverEntryList) => {
0040 |     //     console.log('observer1:entryList.getEntries()' + JSON.stringify(entryList.getEntr...
0041 |     //   }
0042 |     // )
0043 |     // observer1.observe({
0044 |     //   entryTypes: ['render', 'navigation'],
0045 |     // } as PerformanceObserverOptions)
0046 |     // 统计上报 - 应用启动
0047 |     // #ifdef APP-ANDROID || APP-IOS || WEB || MP-WEIXIN || APP-HARMONY
0048 |     uni.report({
0049 |       name: 'uni-app-launch',
0050 |       options: res,
```

```
0051 |     success(res_data) {
0052 |         console.log(res_data);
0053 |     }, fail(err) {
0054 |         console.log(err);
0055 |     }
0056 | })
0057 | // #endif
0058 | // #ifdef APP
0059 | if (process.env.NODE_ENV !== 'development') { //真机运行可以注释此条件
0060 |     uni.getPrivacySetting({
0061 |         success(res1){
0062 |             if(res1.needAuthorization){
0063 |                 uni.openDialogPage({
0064 |                     url: '/pages/component/button/privacy',
0065 |                 })
0066 |             }
0067 |         }
0068 |     })
0069 | }
0070 | // #endif
0071 | // 获取系统主题
0072 | checkSystemTheme();
0073 | },
0074 | onShow: function (res : OnShowOptions) {
0075 |     this.globalData.onShowOption = res
0076 |     // 处理scheme或通用链接直达
0077 |     let url = this.getRedirectUrl(res.appScheme, res.appLink);
0078 |     if (null !== url) {
0079 |         uni.navigateTo({
0080 |             url: url
0081 |         })
0082 |     }
0083 |     // 自动化测试
0084 |     setLifecycleNum(state.lifecycleNum + 100)
0085 |     console.log('App Show')
0086 |     // #ifdef APP-ANDROID || APP-IOS || WEB || MP-WEIXIN || APP-HARMONY
0087 |     // 统计上报 - 应用显示
0088 |     uni.report({
0089 |         name: 'uni-app-show',
0090 |         success(res_data) {
0091 |             console.log(res_data);
0092 |         }, fail(err) {
0093 |             console.log(err);
0094 |         }
0095 |     })
0096 |     // #endif
0097 | },
0098 | onHide: function () {
0099 |     // 自动化测试
0100 |     setLifecycleNum(state.lifecycleNum - 100)
```

```
0101 | console.log('App Hide')
0102 | // #ifdef APP-ANDROID || APP-IOS || WEB || MP-WEIXIN || APP-HARMONY
0103 | // 统计上报 - 应用进入后台
0104 | uni.report({
0105 |     name: 'uni-app-hide',
0106 |     success(res) {
0107 |         console.log(res);
0108 |     }, fail(err) {
0109 |         console.log(err);
0110 |     }
0111 | })
0112 | // #endif
0113 | },
0114 | // #ifdef APP-ANDROID || APP-HARMONY
0115 | onLastPageBackPress: function () {
0116 |     // 自动化测试
0117 |     setLifecycleNum(state.lifecycleNum - 1000)
0118 |     console.log('App LastPageBackPress')
0119 |     if (firstBackTime == 0) {
0120 |         uni.showToast({
0121 |             title: '再按一次退出应用',
0122 |             position: 'bottom',
0123 |         })
0124 |         firstBackTime = Date.now()
0125 |         setTimeout(() => {
0126 |             firstBackTime = 0
0127 |         }, 2000)
0128 |     } else if (Date.now() - firstBackTime < 2000) {
0129 |         firstBackTime = Date.now()
0130 |         uni.exit()
0131 |     }
0132 | },
0133 | onExit() {
0134 |     console.log('App Exit')
0135 | },
0136 | // #endif
0137 | onError(err : any) {
0138 |     console.log('App Error', err)
0139 |     setLifecycleNum(state.lifecycleNum + 100)
0140 |     // console.log('App onError', err)
0141 |     // #ifdef APP-ANDROID || APP-IOS || WEB || MP-WEIXIN || APP-HARMONY
0142 |     // 统计上报 - 应用发生错误
0143 |     uni.report({
0144 |         name: 'uni-app-error',
0145 |         options: err,
0146 |         success(res) {
0147 |             console.log(res);
0148 |         }, fail(err) {
0149 |             console.log(err);
0150 |         }
```

```
0151 |    })
0152 |    // #endif
0153 |  },
0154 |  methods: {
0155 |    increaseLifeCycleNum() {
0156 |      setLifeCycleNum(state.lifeCycleNum + 100)
0157 |      console.log('App increaseLifeCycleNum')
0158 |    },
0159 |    getRedirectUrl(scheme : string | null, ulink : string | null) : string | null {
0160 |      //解析scheme或universal link启动直达页面:
0161 |      //scheme格式: uniappx://redirect/pages/component/view/view?key=value //其中redirect后为页面路径
0162 |      //universal link格式: https://uniappx.dcloud.net.cn/ulink/redirect.html?url=%2Fpages%2Fc...
0163 |      let url : string | null = null;
0164 |      if (null != scheme && scheme.length > 0) {
0165 |        const PATHPRE = 'redirect';
0166 |        let parts : string | null = null;
0167 |        let pos = scheme.search('/');
0168 |        if (pos > 0) {
0169 |          parts = scheme.substring(pos + 2);
0170 |        }
0171 |        if (null != parts && parts.startsWith(PATHPRE)) {
0172 |          url = parts.substring(PATHPRE.length);
0173 |        }
0174 |      } else if (null != ulink && ulink.length > 0) {
0175 |        const PATH = 'ulink/redirect.html';
0176 |        let parts = ulink.split('?');
0177 |        if (parts.length > 1 && parts[0].endsWith(PATH) && parts[1].length > 0) {
0178 |          parts[1].split('&').forEach((e) => {
0179 |            let params = e.split('=');
0180 |            if (params.length > 1 && params[0].length > 0 && params[1].length > 0) {
0181 |              if ('url' == params[0]) {
0182 |                if (null == url) {
0183 |                  url = decodeURIComponent(params[1]);
0184 |                }
0185 |              }
0186 |            }
0187 |          });
0188 |        }
0189 |      }
0190 |      return url;
0191 |    }
0192 |  }
0193 | }
0194 | </script>
0195 | <style>
0196 | /*每个页面公共css */
0197 | @import "../common/uni.css";
0198 | /* #ifdef WEB */
0199 | .uni-top-window uni-tabbar .uni-tabbar {
0200 |   background-color: #fff !important;
```

```
0201 | }
0202 | /* #endif */
0203 | /* #ifdef MP-WEIXIN */
0204 | page {
0205 |   background-color: #eff4;
0206 | }
0207 | /* #endif */
0208 | </style>
0209 | // ===== store/index.uts =====
0210 | export type SafeArea = {
0211 |   top : number,
0212 |   right : number,
0213 |   bottom : number,
0214 |   left : number,
0215 |   width : number,
0216 |   height : number,
0217 | }
0218 | export const AGREE_PRIVACY = 'UNI-PRIVACY-AGREE'
0219 | export type UserInfo = {
0220 |   nickName: string,
0221 |   avatarUrl?: string
0222 | }
0223 | type State = {
0224 |   lifeCycleNum : number,
0225 |   // 状态栏高度
0226 |   statusBarHeight : number,
0227 |   safeArea : SafeArea
0228 |   // 设备像素比
0229 |   devicePixelRatio : number
0230 |   // 事件判断回调
0231 |   eventCallbackNum: number
0232 |   // 是否匹配左侧窗口
0233 |   noMatchLeftWindow:boolean
0234 |   // 当前激活的tab页
0235 |   active: string
0236 |   leftWinActive:string
0237 |   // 是否同意隐私政策
0238 |   agreeToPrivacy: boolean | null
0239 |   // 是否同意截屏
0240 |   allowCapture: boolean,
0241 |   // 是否暗黑主题 (适配web端)
0242 |   isDarkMode: boolean
0243 |   // 是否无网环境
0244 |   netless: boolean,
0245 |   // 登录用户信息
0246 |   userInfo: UserInfo | null
0247 | }
0248 | const getAgreePrivacy = () => {
0249 |   const data = uni.getStorageSync(AGREE_PRIVACY)
0250 |   if (typeof data === 'boolean') {
```

```
0251 |   return data
0252 | }
0253 | if (data == null) {
0254 |   return null
0255 | }
0256 |   return null
0257 |}
0258 | export const state = reactive({
0259 |   lifeCycleNum: 0,
0260 |   statusBarHeight: 0,
0261 |   devicePixelRatio: 1,
0262 |   eventCallbackNum: 0,
0263 |   noMatchLeftWindow: true,
0264 |   active: 'runIndex',
0265 |   leftWinActive: '/pages/run/index',
0266 |   safeArea: {
0267 |     top: 0,
0268 |     right: 0,
0269 |     bottom: 0,
0270 |     left: 0,
0271 |     width: 0,
0272 |     height: 0,
0273 |   },
0274 |   agreeToPrivacy: getAgreePrivacy(),
0275 |   allowCapture: true,
0276 |   isDarkMode:false,
0277 |   netless: false,
0278 |   userInfo: null
0279 | } as State)
0280 | export const setUserInfo = (userInfo : UserInfo | null) => {
0281 |   state.userInfo = userInfo
0282 | }
0283 | export const setLifeCycleNum = (num : number) => {
0284 |   state.lifeCycleNum = num
0285 | }
0286 | export const setEventCallbackNum = (num : number) => {
0287 |   state.eventCallbackNum = num
0288 | }
0289 | export const setStatusBarHeight = (height : number) => {
0290 |   state.statusBarHeight = height
0291 | }
0292 | export const setSafeArea = (value : SafeArea) => {
0293 |   state.safeArea = value
0294 | }
0295 | export const setDevicePixelRatio = (devicePixelRatio : number) => {
0296 |   state.devicePixelRatio = devicePixelRatio
0297 | }
0298 | export const setMatchLeftWindow = (matchLeftWindow:boolean) => {
0299 |   state.noMatchLeftWindow = !matchLeftWindow
0300 | }
```

```
0301 | export const setActive = (tabPage:string) => {
0302 |   state.active = tabPage
0303 | }
0304 | export const setLeftWinActive = (leftWinActive:string) => {
0305 |   state.leftWinActive = leftWinActive
0306 | }
0307 | export const setAgreeToPrivacy = (agree: boolean) => {
0308 |   state.agreeToPrivacy = agree
0309 |   uni.setStorageSync(AGREE_PRIVACY, agree);
0310 | }
0311 | export const setAllowCapture = (allow: boolean) => {
0312 |   state.allowCapture = allow
0313 | }
0314 | // 检查系统主题
0315 | export const checkSystemTheme = () => {
0316 |   // #ifdef WEB || MP
0317 |   const info = uni.getAppBaseInfo();
0318 |   state.isDarkMode = info.hostTheme === 'dark';
0319 |   uni.onHostThemeChange((result) => {
0320 |     state.isDarkMode = result.hostTheme === 'dark';
0321 |   });
0322 |   // #endif
0323 |   // #ifdef APP
0324 |   uni.getSystemInfo({
0325 |     success: (res : GetSystemInfoResult) => {
0326 |       const appTheme = res.appTheme === "auto" ? res.osTheme! : res.appTheme!
0327 |       state.isDarkMode = appTheme.trim() == 'dark';
0328 |     }
0329 |   })
0330 |   uni.onAppThemeChange((res : AppThemeChangeResult) => {
0331 |     state.isDarkMode = res.appTheme.trim() == 'dark';
0332 |   })
0333 |   // #endif
0334 | }
0335 | export const setNetless = (netless: boolean) => {
0336 |   state.netless = netless
0337 | }
0338 | // ===== utils/auth-uts.uts =====
0339 | /**
0340 |  * 认证/错误 JSON 读取。
0341 |  * 原生端 (Kotlin) 用 UTSJSONObject.getString/getAny;
0342 |  * H5 端 uni.request 返回的 res.data / fail err 都是 plain Object, 没有这些方法,
0343 |  * 所以按平台条件编译, H5 直接用属性访问。
0344 |  *
0345 |  * 另外 Android 上 uni.getStorageSync 拿回来的可能是 JSON 字符串 (跟 H5 把对象直接回
0346 |  * 不一样), 所以原生分支里先用 asUtsJo() 尝试 parse, 拿不到再退出。
0347 |  */
0348 | // #ifndef H5
0349 | /**
0350 |  * 尽量把 p 变成 UTSJSONObject:
```

```
0351 | * - 已经是 UTSJSONObject 直接返回;
0352 | * - 是字符串则 JSON.parse;
0353 | * - 其它情况返回 null, 调用方用空值兜底, 避免 ClassCastException。
0354 | */
0355 | function asUtsJo(p : any | null) : UTSJSONObject | null {
0356 |   if (p == null) return null
0357 |   if (p instanceof UTSJSONObject) return p as UTSJSONObject
0358 |   // UTS 字符串一律用 == / != (结构比较)。`===` 在原生端 (Kotlin) 是引用比较,
0359 |   // 动态生成的字符串即使内容一样也会判为不等。
0360 |   if (typeof p == 'string') {
0361 |     const s = `${p}`
0362 |     if (s == "") return null
0363 |     try {
0364 |       const parsed = JSON.parse(s)
0365 |       if (parsed != null && parsed instanceof UTSJSONObject) {
0366 |         return parsed as UTSJSONObject
0367 |       }
0368 |     } catch (_e) {
0369 |       return null
0370 |     }
0371 |   }
0372 |   return null
0373 | }
0374 | // #endif
0375 | export function utsAuthReadToken(p : any | null) : string {
0376 |   if (p == null) return ""
0377 |   // #ifdef H5
0378 |   const obj = p as any
0379 |   const t = obj['token']
0380 |   return t != null ? `${t}` : ""
0381 |   // #endif
0382 |   // #ifndef H5
0383 |   const jo = asUtsJo(p)
0384 |   if (jo == null) return ""
0385 |   const s = jo.getString('token')
0386 |   return s != null ? `${s}` : ""
0387 |   // #endif
0388 | }
0389 | export function utsAuthReadUser(p : any | null) : any | null {
0390 |   if (p == null) return null
0391 |   // #ifdef H5
0392 |   const obj = p as any
0393 |   return obj['user']
0394 |   // #endif
0395 |   // #ifndef H5
0396 |   const jo = asUtsJo(p)
0397 |   if (jo == null) return null
0398 |   return jo.getAny('user')
0399 |   // #endif
0400 | }
```



```
0401 | export function utsErrReadMessage(e : any | null) : string {
0402 |   if (e == null) return ""
0403 |   // #ifdef H5
0404 |   const obj = e as any
0405 |   // 后端错误体是 { message }, uni.request fail 返回 { errMsg, errno },
0406 |   // Error 实例有 message; 统一按优先级取。
0407 |   let m = obj['message']
0408 |   if (m == null) m = obj['errMsg']
0409 |   if (m == null) m = obj['msg']
0410 |   return m != null ? `${m}` : ""
0411 |   // #endif
0412 |   // #ifndef H5
0413 |   // 原生的 catch(e) 里 e 可能是 UTSError / Throwable / string / UTSJSONObject,
0414 |   // 先尝试当 UTSJSONObject 用; 拿不到就从 toString() 里兜个 message
0415 |   const jo = asUtsJo(e)
0416 |   if (jo != null) {
0417 |     const s = jo.getString('message')
0418 |     if (s != null && s != "") return `${s}`
0419 |     const s2 = jo.getString('errMsg')
0420 |     if (s2 != null && s2 != "") return `${s2}`
0421 |   }
0422 |   if (typeof e == 'string') return `${e}`
0423 |   return ""
0424 |   // #endif
0425 | }
0426 | export type UtsStoredUser = {
0427 |   username : string
0428 |   phone : string
0429 | }
0430 | /**
0431 |  * 从 uni.getStorageSync('user') 拿回来的值里抽 username/phone。
0432 |  * 原生端可能是 UTSJSONObject / JSON 字符串 / null, H5 是 plain Object, 这里都兼容。
0433 |  *
0434 |  * 2026-04 改版后不再存 email 字段（老数据已清空）。
0435 |  */
0436 | export function utsUserReadBasic(p : any | null) : UtsStoredUser {
0437 |   const empty : UtsStoredUser = { username: "", phone: "" }
0438 |   if (p == null) return empty
0439 |   // #ifdef H5
0440 |   const obj = p as any
0441 |   const u = obj['username']
0442 |   const ph = obj['phone']
0443 |   return {
0444 |     username: u != null ? `${u}` : "",
0445 |     phone: ph != null ? `${ph}` : "",
0446 |   }
0447 |   // #endif
0448 |   // #ifndef H5
0449 |   const jo = asUtsJo(p)
0450 |   if (jo == null) return empty
```

```
0451 | const u2 = jo.getString('username')
0452 | const ph2 = jo.getString('phone')
0453 | return {
0454 |   username: u2 != null ? `${u2}` : '',
0455 |   phone: ph2 != null ? `${ph2}` : '',
0456 | }
0457 | // #endif
0458 | }
0459 | /** 读后端返回的 devCode (开发/Mock 模式下才有), 方便联调填自动带入 */
0460 | export function utsAuthReadDevCode(p : any | null) : string {
0461 |   if (p == null) return ''
0462 |   // #ifdef H5
0463 |   const obj = p as any
0464 |   const c = obj['devCode']
0465 |   return c != null ? `${c}` : ''
0466 |   // #endif
0467 |   // #ifndef H5
0468 |   const jo = asUtsJo(p)
0469 |   if (jo == null) return ''
0470 |   const s = jo.getString('devCode')
0471 |   return s != null ? `${s}` : ''
0472 |   // #endif
0473 | }
0474 | /**
0475 |  * 从 error 对象里读 statusCode; 拿不到返回 0。
0476 |  * 和 extractErrorMessage 同样多策略: instanceof → stringify/parse 兜底。
0477 |  */
0478 | function utsErrReadStatus(e : any | null) : number {
0479 |   if (e == null) return 0
0480 |   // #ifdef H5
0481 |   const obj = e as any
0482 |   const s = obj['statusCode']
0483 |   if (s == null) return 0
0484 |   if (typeof s == 'number') return s as number
0485 |   const parsed = parseInt(`${s}`)
0486 |   return isNaN(parsed) ? 0 : parsed
0487 |   // #endif
0488 |   // #ifndef H5
0489 |   // 策略 1: instanceof
0490 |   if (e instanceof UTSJSONObject) {
0491 |     const jo = e as UTSJSONObject
0492 |     const n = jo.getNumber('statusCode')
0493 |     if (n != null) {
0494 |       const parsed = parseInt(`${n}`)
0495 |       if (!isNaN(parsed)) return parsed
0496 |     }
0497 |   }
0498 |   // 策略 2: stringify/parse round-trip
0499 |   try {
0500 |     const s = JSON.stringify(e)
```

```
0501 |   if (s != null && s != '' && s != 'null') {
0502 |       const jo2 = JSON.parse<UTSJSONObject>(s)
0503 |       if (jo2 != null) {
0504 |           const n = jo2.getNumber('statusCode')
0505 |           if (n != null) {
0506 |               const parsed = parseInt(`${n}`)
0507 |               if (!isNaN(parsed)) return parsed
0508 |           }
0509 |       }
0510 |   }
0511 | } catch (_e) {}
0512 | return 0
0513 | // #endif
0514 | }
0515 | /**
0516 |  * 把后端原文错误 / 网络异常翻译成用户能看的中文提示。
0517 |  * scene:
0518 |  * 'register' - 注册场景
0519 |  * 'login' - 邮箱登录场景
0520 |  * 'wechat' - 微信登录场景
0521 |  * 其它 - 通用兜底
0522 |  * 原则：不把后端英文原文 / 堆栈 / errorCode 暴露给用户，只给白名单里的友好文案；
0523 |  * 识别不出来就回到场景默认提示。
0524 | */
0525 | /** 不区分大小写的子串包含判断：UTS / Kotlin 下 toLowerCase + indexOf 链在部分
0526 |  * Android 环境行为奇怪（曾遇到 indexOf 返回 >=0 但条件链不走进分支的情况），
0527 |  * 统一用这个帮手包一层，所有比较都先各自 toLowerCase 再判。 */
0528 | function ciContains(hay : string, needle : string) : boolean {
0529 |   if (hay.length == 0 || needle.length == 0) return false
0530 |   const h = hay.toLowerCase()
0531 |   const n = needle.toLowerCase()
0532 |   return h.indexOf(n) >= 0
0533 | }
0534 | /** 多策略提取错误消息：instanceof UTSJSONObject 在原生端时不时不灵，
0535 |  * 我们按顺序尝试：
0536 |  * 1) instanceof UTSJSONObject → getString('message') / ('errMsg')
0537 |  * 2) JSON.stringify + JSON.parse<UTSJSONObject> round-trip 再 getString
0538 |  * 3) toString() 兜底
0539 |  * 任何一步拿到非空字符串立刻返回。 */
0540 | function extractErrorMessage(e : any) : string {
0541 |   // 策略 1：原生就是 UTSJSONObject
0542 |   // #ifndef H5
0543 |   if (e instanceof UTSJSONObject) {
0544 |       const jo = e as UTSJSONObject
0545 |       const m = jo.getString('message')
0546 |       if (m != null && m != '') return `${m}`
0547 |       const em = jo.getString('errMsg')
0548 |       if (em != null && em != '') return `${em}`
0549 |       const msg2 = jo.getString('msg')
0550 |       if (msg2 != null && msg2 != '') return `${msg2}`
```

```
0551 | }
0552 | // 策略 2: stringify -> parse round-trip (instanceof 失灵时的兜底)
0553 | try {
0554 |   const s = JSON.stringify(e)
0555 |   if (s !== null && s !== '' && s !== 'null' && s !== '{}') {
0556 |     const jo2 = JSON.parse<UTSJSONObject>(s)
0557 |     if (jo2 !== null) {
0558 |       const m = jo2.getString('message')
0559 |       if (m !== null && m !== '') return `${m}`
0560 |       const em = jo2.getString('errMsg')
0561 |       if (em !== null && em !== '') return `${em}`
0562 |     }
0563 |   }
0564 | } catch (_e) {}
0565 | // #endif
0566 | // #ifdef H5
0567 | const obj = e as any
0568 | const m1 = obj['message']
0569 | if (m1 !== null && `${m1}` !== '') return `${m1}`
0570 | const m2 = obj['errMsg']
0571 | if (m2 !== null && `${m2}` !== '') return `${m2}`
0572 | const m3 = obj['msg']
0573 | if (m3 !== null && `${m3}` !== '') return `${m3}`
0574 | // #endif
0575 | // 策略 3: toString 兜底
0576 | const fallback = `${e}`
0577 | return fallback !== '' && fallback !== '[object Object]' && fallback !== 'null' ? fallback : ''
0578 | }
0579 | export function friendlyAuthError(e : any | null, scene : string) : string {
0580 |   const defaultMsg =
0581 |     scene == 'register' ? '注册失败, 请稍后重试'
0582 |     : scene == 'login' ? '登录失败, 请稍后重试'
0583 |     : scene == 'sms' ? '短信发送失败, 请稍后重试'
0584 |     : scene == 'reset' ? '重置密码失败, 请稍后重试'
0585 |     : scene == 'bind' ? '绑定手机号失败, 请稍后重试'
0586 |     : scene == 'wechat' ? '微信登录失败, 请稍后重试'
0587 |     : '操作失败, 请稍后重试'
0588 |   if (e == null) {
0589 |     console.log(`[friendlyAuthError] e == null, return defaultMsg`)
0590 |     return defaultMsg
0591 |   }
0592 |   const status = utsErrReadStatus(e)
0593 |   const raw = extractErrMsg(e).trim()
0594 |   console.log(`[friendlyAuthError] scene=${scene} + status=${status} + raw=${raw} + ...
0595 | // ===== 后端中文错误体: 直接回显 (它们本来就是写给用户看的) =====
0596 | // 新版后端所有 4xx 返回中文 message, 通过这条分支原样展示, UX 一致。
0597 | // 兜底条件: raw 非空、包含中文或明显是短提示 (<= 40 字符)、状态码是 4xx。
0598 | if (raw.length > 0 && status >= 400 && status < 500) {
0599 |   // 有中文字符基本就是后端定制文案
0600 |   const hasChinese = /[\\u4e00-\\u9fa5]/.test(raw)
```

```
0601 |   if (hasChinese) return raw
0602 | }
0603 | // ===== 业务语义（英文关键字兜底） =====
0604 | if (raw.indexOf('已注册') >= 0 || raw.indexOf('已被') >= 0) {
0605 |   return raw // 后端就是中文，直接透传
0606 | }
0607 | if (ciContains(raw, 'invalid credentials') || ciContains(raw, 'incorrect password')) {
0608 |   return '账号或密码错误'
0609 | }
0610 | if (ciContains(raw, 'user not found') || ciContains(raw, 'account not exist')) {
0611 |   return scene == 'login' ? '账号或密码错误' : '账号不存在'
0612 | }
0613 | if (ciContains(raw, 'wechat code') || ciContains(raw, 'wechat user info')) {
0614 |   return '微信登录失败，请稍后重试'
0615 | }
0616 | // ===== 网络层：超时 / 不可达 / DNS / 连接失败 =====
0617 | if (ciContains(raw, 'timeout') || raw.indexOf('超时') >= 0) {
0618 |   return '网络连接超时，请检查网络后重试'
0619 | }
0620 | if (
0621 |   raw.indexOf('网络不可达') >= 0 ||
0622 |   ciContains(raw, 'network') ||
0623 |   ciContains(raw, 'unreachable') ||
0624 |   ciContains(raw, 'request:fail') ||
0625 |   ciContains(raw, 'connect')
0626 | ) {
0627 |   return '网络连接失败，请检查网络后重试'
0628 | }
0629 | // ===== HTTP 5xx / 空响应 =====
0630 | if (status >= 500 || ciContains(raw, 'server error') || raw.indexOf('空响应') >= 0) {
0631 |   return '服务暂时不可用，请稍后再试'
0632 | }
0633 | // ===== HTTP 401 / 403 / 404 =====
0634 | if (status == 401) {
0635 |   return scene == 'login' ? '账号或密码错误' : '登录已失效，请重新登录'
0636 | }
0637 | if (status == 403) {
0638 |   return '没有访问权限'
0639 | }
0640 | if (status == 404) {
0641 |   return scene == 'login' ? '账号或密码错误' : '请求的资源不存在'
0642 | }
0643 | if (status == 429) {
0644 |   return raw != '' ? raw : '请求太频繁，请稍后再试'
0645 | }
0646 | // 后端有 message 就用它，没有就默认
0647 | if (raw != '') return raw
0648 | return defaultMsg
0649 | }
0650 | export function utsWxLoginReadCode(res : any | null) : string {
```

```
0651 | if (res == null) return ""
0652 | // #ifdef H5
0653 | const obj = res as any
0654 | const c = obj['code']
0655 | return c != null ? `${c}` : ""
0656 | // #endif
0657 | // #ifndef H5
0658 | const jo = asUtsJo(res)
0659 | if (jo == null) return ""
0660 | const s = jo.getString('code')
0661 | return s != null ? `${s}` : ""
0662 | // #endif
0663 | }
0664 | // ===== utils/env.uts =====
0665 | // 统一的"是否开发构建"标记。
0666 | //
0667 | // 约定：
0668 | // - 普通运行（pnpm dev / 微信开发者工具 / HBuilderX 调试）时为 true
0669 | // - 打包发布（UNI_NODE_ENV=production）时为 false
0670 | //
0671 | // 为什么这么写：
0672 | // uni-app 的构建链里，`process.env.NODE_ENV` 在 uvue 编译时会被替换成字面量
0673 | // 'development' / 'production'；但为了兼容个别早期版本取不到 process 的情况，
0674 | // 这里再做一层防御，取不到就回退到 true（宁可在测试里显示调试入口，也不要
0675 | // 在真实用户那露出）。
0676 | // 如果以后你要在真机联调时也隐藏调试入口，直接改这里一个常量即可。
0677 | function detectIsDev(): boolean {
0678 |   try {
0679 |     // #ifdef H5
0680 |     // H5 一定能读到 process.env.NODE_ENV（构建期注入）
0681 |     // @ts-ignore
0682 |     const nodeEnv = `${process.env.NODE_ENV != null ? process.env.NODE_ENV : ""}`
0683 |     return nodeEnv != 'production'
0684 |     // #endif
0685 |     // #ifndef H5
0686 |     // App/小程序端：uni-app-x 也会在编译期替换，但为了更稳妥，这里再做 fallback
0687 |     // @ts-ignore
0688 |     const nodeEnv2 = `${process.env.NODE_ENV != null ? process.env.NODE_ENV : ""}`
0689 |     if (nodeEnv2 == 'production') return false
0690 |     return true
0691 |     // #endif
0692 |   } catch (_e) {
0693 |     return true
0694 |   }
0695 | }
0696 | export const IS_DEV: boolean = detectIsDev()
0697 | // ===== utils/music-app.uts =====
0698 | /**
0699 | * 点击平台图标 → 拉起对应音乐 App 并跳转到这首歌的播放页（带 songId 时）或搜索页。
0700 | *
```

```
0701 | * 两个入口：
0702 | * 1) openSongInMusicApp(key, title, artist) ← 推荐，带歌名+歌手，走后端解析歌曲 ID
0703 | * 前端先 GET /api/music/resolve 拿到 deeplink，再 openSchema 拉起 App，
0704 | * 失败（后端挂了 / 没搜到）自动降级到 openInMusicApp 的"拉首页 + 剪贴板"行为。
0705 | *
0706 | * 2) openInMusicApp(key, query) ← 老入口，只拉首页 + 复制到剪贴板
0707 | * 保留兼容，内部实现不变。
0708 | *
0709 | * 2026 说明：
0710 | * - Apple Music / Spotify：后端用官方 API 解析，稳定，直达歌曲页
0711 | * - 网易云 / QQ 音乐：用非官方但公开的搜索接口，通常能拿到 ID 直达歌曲页
0712 | * - 酷狗：拿到 ID 后用 https Universal Link 触发，需设备装了对应 App；
0713 | * 外部唤起时播放栏会显示"playone / 未知歌手"（酷狗限制，需官方分享
0714 | * 签名才显示真实 meta），用 showKugouMetaHintOnce() 一次性说明
0715 | *
0716 | * 曾支持酷我、汽水，均已移除（见 backend/music-resolve.service.ts 顶部注释）。
0717 | */
0718 | // #ifdef APP-ANDROID || APP-IOS || APP-HARMONY
0719 | import { openSchema, canOpenURL, openSchemaForce, launchAppByPackage, openSchemaInPackage } f...
0720 | // #endif
0721 | import { runResolveMusicDeeplink } from '@/utils/api'
0722 | import { runJoStr, runJoSub, runJoArr, runJoOptInt } from '@/utils/run-uts.uts'
0723 | export type MusicPlatform = 'qq' | 'netease' | 'kugou' | 'apple' | 'spotify'
0724 | export type MusicPlatformMeta = {
0725 |   key: MusicPlatform
0726 |   label: string
0727 |   // 图标用一个 emoji 或中文缩写占位（不引入外部资源）
0728 |   icon: string
0729 |   // 色块配色
0730 |   color: string
0731 | }
0732 | export function musicPlatformList(): MusicPlatformMeta[] {
0733 |   // Android 端不展示 Apple Music / Spotify：
0734 |   // - Apple Music Android 版国内用户覆盖率极低（需美区/港区 Apple ID + 付费
0735 |   // 订阅），即便装了，分享出去的 deeplink 也常因 Universal Link 未
0736 |   // autoVerify 跳到浏览器页
0737 |   // - Spotify 在中国大陆无法正常使用（风控 + 付费），展示等于误导
0738 |   // iOS / Harmony / H5 / 小程序端仍保留两项，这些平台有真实用户场景。
0739 |   // #ifdef APP-ANDROID
0740 |   return [
0741 |     { key: 'netease', label: '网易云', icon: '云', color: '#EF4444' },
0742 |     { key: 'qq', label: 'QQ 音乐', icon: 'QQ', color: '#22C55E' },
0743 |     { key: 'kugou', label: '酷狗', icon: '酷', color: '#60A5FA' },
0744 |   ]
0745 |   // #endif
0746 |   // #ifndef APP-ANDROID
0747 |   return [
0748 |     { key: 'netease', label: '网易云', icon: '云', color: '#EF4444' },
0749 |     { key: 'qq', label: 'QQ 音乐', icon: 'QQ', color: '#22C55E' },
0750 |     { key: 'kugou', label: '酷狗', icon: '酷', color: '#60A5FA' },
```

```
0751 |   { key: 'apple', label: 'Apple Music', icon: '', color: '#F472B6' },
0752 |   { key: 'spotify', label: 'Spotify', icon: 'Sp', color: '#1DB954' },
0753 | ]
0754 | // #endif
0755 | }
0756 | /** 拼 "歌名 歌手" 作为搜索关键词（复制到剪贴板时用） */
0757 | export function buildMusicQuery(title : string, artist : string) : string {
0758 |   const t = title.trim()
0759 |   const a = artist.trim()
0760 |   if (a == '') return t
0761 |   return `${t} ${a}`
0762 | }
0763 | /** 该平台点完后，是否需要提示用户"到搜索框粘贴"（只有国内三家的降级路径才提示） */
0764 | function needsPasteHint(key : MusicPlatform) : boolean {
0765 |   return key == 'qq' || key == 'netease' || key == 'kugou'
0766 | }
0767 | /**
0768 |  * 酷狗「直达详情页」的已知限制 —— 首次跳转时给用户一次性说明。
0769 |  *
0770 |  * 现象：即便深链带上了 songname / singername / filename 等所有展示字段，
0771 |  * 酷狗 App 播放栏仍会把歌名显示成内部常量`playone`、歌手显示成`未知歌手`。
0772 |  * 实测这是酷狗`start.weixin` Activity 的硬编码行为 —— 它只信任来自
0773 |  * 微信客户端的合法分享签名（含`scene`/`unicode`等字段），对 URL query
0774 |  * 传过来的展示参数一律丢弃。没有官方 SDK / BD 合作就改不了。
0775 |  *
0776 |  * 实际播放内容是正确的（`hash + album_audio_id`都带上了），只是 UI
0777 |  * 文案错。这里用 showModal 给用户解释清楚，避免误解为"播错歌了"。
0778 |  */
0779 | const KUGOU_META_HINT_KEY = 'kugou_meta_hint_shown_v1'
0780 | function showKugouMetaHintOnce() : void {
0781 |   try {
0782 |     const shown = uni.getStorageSync(KUGOU_META_HINT_KEY)
0783 |     if (shown != null && `${shown}` != '') return
0784 |     uni.setStorageSync(KUGOU_META_HINT_KEY, '1')
0785 |     uni.showModal({
0786 |       title: '已在酷狗播放',
0787 |       content: '提示：酷狗 App 外部调起时，播放栏可能显示为「playone / 未知歌手」，这是酷狗自身的限制（需官方分享签名才显示真实歌名）。实际播放的正是您选中...',
0788 |       showCancel: false,
0789 |       confirmText: '知道了',
0790 |       success: () => {},
0791 |       fail: () => {},
0792 |     })
0793 |   } catch () {}
0794 | }
0795 | function platformLabel(key : MusicPlatform) : string {
0796 |   if (key == 'qq') return 'QQ 音乐'
0797 |   if (key == 'netease') return '网易云音乐'
0798 |   if (key == 'kugou') return '酷狗音乐'
0799 |   if (key == 'apple') return 'Apple Music'
0800 |   if (key == 'spotify') return 'Spotify'
```



```
0801 | return '音乐 App'
0802 | }
0803 | /**
0804 | * 各家音乐 App 的 Android 包名 —— 当后端 /resolve 整个挂掉、拿不到
0805 | * packageName 时的纯前端兜底列表。须与 AndroidManifest.xml 的 <queries> 保持
0806 | * 同步，否则 launchAppByPackage 会查不到。
0807 | */
0808 | function platformAndroidPackages(key : MusicPlatform) : string[] {
0809 |   if (key == 'qq') return ['com.tencent.qqmusic']
0810 |   if (key == 'netease') return ['com.netease.cloudmusic']
0811 |   if (key == 'kugou') return ['com.kugou.android', 'com.kugou.android.lite', 'com.kugou.andro...
0812 |   // Apple Music / Spotify 在国内用户 Android 端基本用不到（且 Apple Music
0813 |   // Android 的 scheme 和 Universal Link 本身就工作良好），不列在里面。
0814 |   return []
0815 | }
0816 | /** 首页 scheme / 网页搜索页 —— 拿不到歌曲 ID 时兜底用 */
0817 | function homeScheme(key : MusicPlatform) : string {
0818 |   if (key == 'qq') return 'qqmusic://'
0819 |   if (key == 'netease') return 'orpheus://'
0820 |   // 酷狗最新版实测 `kugou://` 和 `kugouURL://` 都拿不到 activity，scheme 路径
0821 |   // 不稳定；返回空字符串，让上层直接走 m.kugou.com 的移动页面
0822 |   if (key == 'kugou') return ''
0823 |   // Apple Music 的 `music://` 自定义 scheme 只能唤起 App，
0824 |   // **不解析 /search 这种业务路径**——结果就是 App 起来后停在上次的首页。
0825 |   // 用 https Universal Link 让 iOS 走 applinks 路由，Apple Music 才会真正
0826 |   // 跳进搜索页（query 由调用方拼接到 webSearchUrl）。
0827 |   if (key == 'apple') return ''
0828 |   if (key == 'spotify') return 'spotify://'
0829 |   return ''
0830 | }
0831 | function webSearchUrl(key : MusicPlatform, q : string) : string {
0832 |   const enc = encodeURIComponent(q)
0833 |   if (key == 'netease') return `https://music.163.com/#/search/m/?s=${enc}&type=1`
0834 |   if (key == 'qq') return `https://y.qq.com/n/ryqq/search?w=${enc}&t=song`
0835 |   // 酷狗优先用移动版搜索页 —— 页面顶部自带“在App中打开”横幅
0836 |   if (key == 'kugou') return `https://m.kugou.com/v2/static/search.html?keyword=${enc}`
0837 |   if (key == 'apple') return `https://music.apple.com/cn/search?term=${enc}`
0838 |   if (key == 'spotify') return `https://open.spotify.com/search/${enc}`
0839 |   return ''
0840 | }
0841 | /**
0842 | * 是否是已知的私有 scheme（manifest 白名单里的）。
0843 | * 对这些 scheme 要绕过 canOpenURL —— Android 11+ 的 package visibility 会让
0844 | * canOpenURL 恒假，必须直接 startActivity 才知道 App 是否真的装着。
0845 | */
0846 | function isKnownPrivateScheme(url : string) : boolean {
0847 |   if (url == '') return false
0848 |   return url.indexOf('qqmusic:') == 0
0849 |     || url.indexOf('orpheus:') == 0
0850 |     || url.indexOf('kugou:') == 0
```

```
0851 |   || url.indexOf('kugouURL:') == 0
0852 |   || url.indexOf('kgmusic:') == 0
0853 |   || url.indexOf('kugouktv:') == 0
0854 |   || url.indexOf('music:') == 0
0855 |   || url.indexOf('spotify:') == 0
0856 |   || url.indexOf('itms-music:') == 0
0857 |   || url.indexOf('itmss:') == 0
0858 | }
0859 | /**
0860 |  * 后端解析失败时的兜底：拉起 App 首页（或网页搜索），把歌名复制到剪贴板。
0861 |  * 这就是老的 openInMusicApp 的行为，抽成内部函数。
0862 |  */
0863 | /**
0864 |  * Apple Music 专用「未找到歌曲」提示：仅 toast + 复制歌名，**不**拉起 Apple Music。
0865 |  *
0866 |  * 为什么 Apple 这条不走通用的 fallbackOpenHomeWithClipboard：
0867 |  * - 通用兜底会拉 App 主页。但 Apple Music 的 search Universal Link 不会被
0868 |  *   App 当 search 路径解析（实测见 https://music.apple.com/cn/search?term=...
0869 |  *   在 iPhone 上只是把 App 拉到首页，term 参数被吞）；
0870 |  * - "拉起 App 后停在首页" 正是用户最初抱怨的"点苹果按钮只拉首页"行为，
0871 |  *   在我们已经知道拿不到 trackId 的前提下再做这件事是欺骗 —— 用户以为
0872 |  *   App 出问题了；
0873 |  * - 老老实实告诉用户"找不到这首歌"，把歌名写到剪贴板让他自己进 App 搜
0874 |  *   是对他更尊重的处理。
0875 |  *
0876 |  * 触发条件（都在 openSongInMusicApp 的 apple 分支里）：
0877 |  * 1) resolveAppleClientSide 失败 + 后端 /api/music/resolve 也只能给 search
0878 |  *   fallback (songId 为空) —— 上下游 iTunes API 都不可达 / 都没搜到；
0879 |  * 2) 后端 resolve 整体挂了（网络断、后端 500）；
0880 |  * 3) 拿到了 deepLink/webUrl 但 openDeepLink 全部失败（极罕见，仅当 iOS
0881 |  *   连 https://music.apple.com 都拒绝交给 Safari 时才会发生）。
0882 |  */
0883 | function appleNoMatchToast(query : string) : void {
0884 |   if (query != '') {
0885 |     uni.setClipboardData({
0886 |       data: query,
0887 |       success: () => {},
0888 |       fail: () => {},
0889 |     })
0890 |   }
0891 |   uni.showToast({
0892 |     title: '未找到此歌曲，歌名已复制',
0893 |     icon: 'none',
0894 |     duration: 2500,
0895 |   })
0896 | }
0897 | function fallbackOpenHomeWithClipboard(key : MusicPlatform, query : string) : void {
0898 |   if (query == '') return
0899 |   // 永远先把歌名复制到剪贴板 —— 国内几家 App 里只能搜索时粘贴
0900 |   uni.setClipboardData({
```

```
0901 |   data: query,
0902 |   success: () => {},
0903 |   fail: () => {},
0904 | })
0905 | const home = homeScheme(key)
0906 | const web = webSearchUrl(key, query)
0907 | const name = platformLabel(key)
0908 | // #ifdef APP-ANDROID || APP-IOS || APP-HARMONY
0909 | let opened = false
0910 | try {
0911 |   if (home !== '') {
0912 |     // 同 openDeepLink: 对私有 scheme 走 Force, 由 startActivity 判断真实装没装
0913 |     if (isKnownPrivateScheme(home)) {
0914 |       opened = openSchemaForce(home)
0915 |     } else if (canOpenURL(home)) {
0916 |       openSchema(home)
0917 |       opened = true
0918 |     }
0919 |   }
0920 | } catch (_e) {
0921 |   opened = false
0922 | }
0923 | // Android: scheme 不行就按包名拉起 (launchIntent), 最稳。iOS 上 launchAppByPackage 恒 false。
0924 | if (!opened) {
0925 |   const pkgs = platformAndroidPackages(key)
0926 |   for (let i = 0; i < pkgs.length; i++) {
0927 |     if (launchAppByPackage(pkgs[i])) {
0928 |       opened = true
0929 |       break
0930 |     }
0931 |   }
0932 | }
0933 | if (!opened) {
0934 |   try { openSchema(web) } catch (_e) {}
0935 | }
0936 | if (needsPasteHint(key)) {
0937 |   uni.showToast({
0938 |     title: opened ? `歌名已复制, 到${(name)}搜索框粘贴` : `未装${(name)}, 已打开网页搜索`,
0939 |     icon: 'none',
0940 |     duration: 2500,
0941 |   })
0942 | } else {
0943 |   uni.showToast({
0944 |     title: opened ? `正在打开${(name)}...` : `${(name)} 未安装, 已打开网页`,
0945 |     icon: 'none',
0946 |     duration: 2000,
0947 |   })
0948 | }
0949 | // #endif
0950 | // #ifdef H5 || WEB
```

```
0951 | if (key == 'apple' || key == 'spotify') {
0952 |   // 海外平台：H5 没原生 App 可拉，统一开新标签页跳网页版 ——
0953 |   // 用 location.href 替换当前页面会冲掉 dev 预览，开新窗口对开发体验更友好。
0954 |   // 弹窗被浏览器拦截时 window.open 返回 null，回退到当前页跳转。
0955 |   // @ts-ignore window 仅 H5 才有
0956 |   const w1 = window.open(web, '_blank')
0957 |   if (w1 == null) {
0958 |     // @ts-ignore
0959 |     window.location.href = web
0960 |   }
0961 |   uni.showToast({ title: `正在跳转到${name}...`, icon: 'none' })
0962 |   return
0963 | }
0964 | try {
0965 |   // 国内三家：先开新窗口尝试拉起 App 私有 scheme，1.5s 后兜底跳网页搜索页。
0966 |   // 同样改成新窗口，保留 dev 预览不丢上下文。
0967 |   // @ts-ignore
0968 |   const winHome = window.open(home, '_blank')
0969 |   setTimeout(() => {
0970 |     // @ts-ignore
0971 |     const winWeb = window.open(web, '_blank')
0972 |     if (winWeb == null) {
0973 |       // @ts-ignore
0974 |       window.location.href = web
0975 |     }
0976 |   }, 1500)
0977 |   if (winHome == null) {
0978 |     // 浏览器把第一个 window.open 拦了 —— 降级回当前页直接跳网页
0979 |     // @ts-ignore
0980 |     window.location.href = web
0981 |   }
0982 | } catch (_) {
0983 |   // @ts-ignore
0984 |   window.location.href = web
0985 | }
0986 | uni.showToast({ title: '歌名已复制，到搜索框粘贴', icon: 'none', duration: 2500 })
0987 | // #endif
0988 | // #ifdef MP-WEIXIN || MP-ALIPAY || MP-BAIDU || MP-TOUTIAO || MP-QQ || MP-KUAISHOU || MP-JD...
0989 | uni.showModal({
0990 |   title: '歌名已复制',
0991 |   content: `请打开${name}，到搜索框粘贴即可播放。`,
0992 |   showCancel: false,
0993 |   confirmText: '好的',
0994 | })
0995 | // #endif
0996 | }
0997 | /**
0998 | * 用拿到的 deepLink 实际拉起 App；失败返回 false（让上层走兜底）。
0999 | *
1000 | * Android 关键点：
```

```
1001 | * `canOpenURL` 在 Android 11+ 上对于未在 AndroidManifest `<queries>` 中声明
1002 | * 的目标包会恒返回 false（uniapp manifest 里的 urlschemewhitelist 只对 iOS
1003 | * 有效），所以不能再用它当“App 装没装”的判据——否则就会出现 App 明明装
1004 | * 着、scheme 也对、但系统始终跳浏览器的现象（这就是早先“QQ/酷狗 跳浏览
1005 | * 器”的根因）。
1006 | * 对白名单里的已知私有 scheme 直接 `openSchemaForce`，由 startActivity 的
1007 | * ActivityNotFoundException 来判断“真的未安装”。
1008 | *
1009 | * H5 用 location.href 尝试，浏览器自己会弹“打开 App”的提示。
1010 | */
1011 | function openDeepLink(deepLink : string) : boolean {
1012 |   if (deepLink == '') return false
1013 |   // #ifdef APP-ANDROID || APP-IOS || APP-HARMONY
1014 |   try {
1015 |     const isHttps = deepLink.indexOf('https://') == 0 || deepLink.indexOf('http://') == 0
1016 |     // https 和已知私有 scheme 都走 Force：前者由浏览器/已注册 App 兜底，后者
1017 |     // 绕过 canOpenURL 假阴性
1018 |     if (isHttps || isKnownPrivateScheme(deepLink)) {
1019 |       return openSchemaForce(deepLink)
1020 |     }
1021 |     // 未知 scheme 保守走 canOpenURL 校验
1022 |     if (canOpenURL(deepLink)) {
1023 |       openSchema(deepLink)
1024 |       return true
1025 |     }
1026 |     return false
1027 |   } catch (_e) {
1028 |     return false
1029 |   }
1030 |   // #endif
1031 |   // #ifdef H5 || WEB
1032 |   try {
1033 |     // 优先开新标签页：私有 scheme（music:// / spotify://）浏览器会忽略不响应
1034 |     // 但 https URL 会真打开一个新页签，dev 预览不被冲掉。
1035 |     // 弹窗被拦时 window.open 返回 null，对私有 scheme 不退当前页（避免显示
1036 |     // “无法访问 music://”的报错页），只对 https 退当前页跳转。
1037 |     // @ts-ignore window 仅 H5 才有
1038 |     const w = window.open(deepLink, '_blank')
1039 |     if (w != null) return true
1040 |     if (deepLink.indexOf('https://') == 0 || deepLink.indexOf('http://') == 0) {
1041 |       // @ts-ignore
1042 |       window.location.href = deepLink
1043 |       return true
1044 |     }
1045 |     return false
1046 |   } catch (_e) {
1047 |     return false
1048 |   }
1049 |   // #endif
1050 |   // #ifdef MP-WEIXIN || MP-ALIPAY || MP-BAIDU || MP-TOUTIAO || MP-QQ || MP-KUAISHOU || MP-JD...
```

```
1051 | return false
1052 | // #endif
1053 | }
1054 | /**
1055 | * 按包名拉起 App（Android 专用终极兜底）。返回 true 表示成功。
1056 | * 注意：必须在 AndroidManifest.xml 的 <queries> 里声明过 pkg，否则
1057 | * getLaunchIntentForPackage 在 Android 11+ 上恒返回 null。
1058 | * iOS / Harmony / Web 上 launchAppByPackage 本身就恒返回 false，顺着调就行。
1059 | *
1060 | * 参数类型写成 `any | null` 而不是 `any[] | null`：`runJoArr` 的返回类型就是
1061 | * `any | null`（UTS 里 `any[]` 会被当成 `UTSArray<Any>`，和 `Any?` 不兼容，
1062 | * 直接传会编译报 error17）。内部判空 + 强转再用。
1063 | */
1064 | function tryLaunchByPackages(packagesRaw : any | null) : boolean {
1065 |   if (packagesRaw == null) {
1066 |     console.log('[music-app] tryLaunchByPackages: packagesRaw is null')
1067 |     return false
1068 |   }
1069 |   // #ifdef APP-ANDROID || APP-IOS || APP-HARMONY
1070 |   try {
1071 |     const arr = packagesRaw as any[]
1072 |     console.log('[music-app] tryLaunchByPackages: count=', arr.length)
1073 |     for (let i = 0; i < arr.length; i++) {
1074 |       const pkg = `${arr[i]}`
1075 |       if (pkg != '' && launchAppByPackage(pkg)) return true
1076 |     }
1077 |   } catch (e) {
1078 |     console.warn('[music-app] tryLaunchByPackages threw:', e)
1079 |   }
1080 |   // #endif
1081 |   return false
1082 | }
1083 | /**
1084 | * 对每条 URL（HTTPS 或私有 scheme）× 每个包名做笛卡尔积，调用
1085 | * `openSchemaInPackage(url, pkg)` —— 这是“直达歌曲详情页”的关键一步：
1086 | *
1087 | * - Android: `Intent.setPackage(pkg) + ACTION_VIEW(Uri.parse(url))`。
1088 | *   setPackage 后 intent 匹配规则放宽，不再要求 CATEGORY_BROWSABLE，
1089 | *   即使目标 App 的 intent-filter 没对外公开，也能命中内部 Activity。
1090 | * - iOS/Harmony/Web: 回退到 openSchemaForce(url)（按包拉起对它们无意义）
1091 | *
1092 | * URL 列表优先传 `inAppSchemes`（往往能带歌曲 ID 直达），然后是 `webUrl`
1093 | *（HTTPS 域名，目标 App 的 App Link intent-filter 处理后多能直达详情）。
1094 | */
1095 | function tryOpenInPackages(urls : string[], packagesRaw : any | null) : boolean {
1096 |   if (urls.length == 0) return false
1097 |   if (packagesRaw == null) return false
1098 |   // #ifdef APP-ANDROID || APP-IOS || APP-HARMONY
1099 |   try {
1100 |     const pkgs = packagesRaw as any[]
```

```
1101 |   for (let u = 0; u < urls.length; u++) {
1102 |     const url = urls[u]
1103 |     if (url == '') continue
1104 |     for (let p = 0; p < pkgs.length; p++) {
1105 |       const pkg = `${pkgs[p]}`
1106 |       if (pkg != '' && openSchemaInPackage(url, pkg)) return true
1107 |     }
1108 |   }
1109 | } catch (e) {
1110 |   console.warn('[music-app] tryOpenInPackages threw:', e)
1111 | }
1112 | // #endif
1113 | return false
1114 |}
1115 |/**
1116 | * 客户端 (用户终端) 直连 iTunes Search API 拿 trackId, 绕开后端国内出网失败。
1117 | *
1118 | * 为什么要做这层: 后端 dev 机在国内直连 itunes.apple.com 经常 ECONNRESET (GFW
1119 | * 注入 RST) —— 但用户手机的蜂窝/Wi-Fi 出口一般是通的 (AppStore 走的就是这
1120 | * 个域)。「用户终端能连, 开发后端连不上」是国内开发的常态。命中即可拼出
1121 | * 「直达单曲页」的 deepLink, 跳过搜索页兜底。
1122 | *
1123 | * 4s 超时: iTunes Search 90% 命中在 1s 内; 再长会让 loading 体感拖沓, 远不如
1124 | * 直接退到后端搜索页快。
1125 | *
1126 | * miss 的几种情况 (上层都退到后端兜底):
1127 | * - title 为空
1128 | * - 网络失败 / 超时 (同 dev box 网络环境的设备)
1129 | * - H5 浏览器端: itunes.apple.com 不返 CORS 头, 浏览器会拦掉 (恒失败)
1130 | * - 接口返回但 results 为空 (这首歌 cn 区曲库没有)
1131 | * - 没解析出 trackId
1132 | *
1133 | * 返回结构 (永远 resolve 同一种 shape, 与后端 ResolvedSong 同形):
1134 | * { deepLink: string, webUrl: string, songId: string }
1135 | *
1136 | * - **songId 非空** = 命中, 可以直接拼 deepLink 拉起 App
1137 | * - **songId 为空** = miss, 整个对象其它字段也是空字符串, 上层判 songId 决定走哪条兜底
1138 | *
1139 | * - webUrl = trackViewUrl (iTunes 给的官方 Universal Link, 形如
1140 | *       https://music.apple.com/cn/album/foo/1234?i=5678)
1141 | * - deepLink = music:// + 同路径 (单曲详情页 music:// scheme 是 OK 的,
1142 | *       与 search 路径不同, Apple Music custom-scheme handler 认)
1143 | *
1144 | * 实现注意 (**踩过的坑**, 改之前看清楚):
1145 | * - `new Promise((resolve, reject) => {...})` 不带泛型 —— uni-app-x 的 UTS
1146 | *   编译器对 `new Promise<T>(...)` 的支持有问题 (会触发"条件编译失败", 与
1147 | *   api.ts 里 request() 的写法保持一致)。返回值类型也用 `Promise<any>`。
1148 | * - **永远不要 `resolve(null)`**: H5/iOS 端弱类型不报错, 但 Android Kotlin
1149 | *   编译器认 `Promise<any>` 为 `Promise<Any>` (non-null), `resolve(null)` 会被
1150 | *   拒, 错误形如: `Null cannot be a value of a non-null type '@ParameterName(...) Any'`。
```

```
1151 | * 用 songId=" 当 miss 信号即可, 不用 null。
1152 | * - `uni.request<any>({...})` 带 `` 泛型, 同样和 api.ts 对齐。
1153 | */
1154 | function resolveAppleClientSide(
1155 |   title : string,
1156 |   artist : string,
1157 | ) : Promise<any> {
1158 |   // miss 时永远 resolve 这个对象 (不能 resolve(null), 会爆 Android Kotlin 编译;
1159 |   // 上层用 runJoStr(res, 'songId') == " 识别 miss)
1160 |   const empty = { deepLink: "", webUrl: "", songId: "" }
1161 |   return new Promise((resolve, _reject) => {
1162 |     const t = title.trim()
1163 |     const a = artist.trim()
1164 |     if (t == "") {
1165 |       resolve(empty)
1166 |       return
1167 |     }
1168 |     const term = a != "" ? `${t} ${a}` : t
1169 |     // 直接把 query 拼到 url 里, 不走 data 参数 ——
1170 |     // uni-app-x 原生端 uni.request 的 data 必须是 UTSJSONObject,
1171 |     // 普通对象字面量会抛 errCode 600008 "data parameter type is invalid"
1172 |     // (见 api.ts 里 request() 的 data 归一化注释) 。
1173 |     const itunesUrl = 'https://itunes.apple.com/search?term=' + encodeURIComponent(term)
1174 |       + '&media=music&entity=song&limit=1&country=cn'
1175 |     uni.request<any>({
1176 |       url: itunesUrl,
1177 |       method: 'GET',
1178 |       timeout: 4000,
1179 |       success: (res) => {
1180 |         try {
1181 |           const data = res.data
1182 |           if (data == null) { resolve(empty); return }
1183 |           const results = runJoArr(data, 'results')
1184 |           if (results == null) { resolve(empty); return }
1185 |           const arr = results as any[]
1186 |           if (arr.length == 0) { resolve(empty); return }
1187 |           const r = arr[0]
1188 |           const trackIdN = runJoOptInt(r, 'trackId')
1189 |           if (trackIdN == null) { resolve(empty); return }
1190 |           const trackId = `${trackIdN}`
1191 |           let webUrl = runJoStr(r, 'trackViewUrl')
1192 |           if (webUrl == "") {
1193 |             webUrl = `https://music.apple.com/cn/song/${trackId}`
1194 |           }
1195 |           // 单曲详情页 music:// 是 OK 的 (与 search 路径不同), 保留给装了
1196 |           // Apple Music 的用户做"App 直跳"; webUrl 当 https 兜底 (通过
1197 |           // Universal Link 路由也能进 App, 未装则进 Safari) 。
1198 |           let deepLink = webUrl
1199 |           if (webUrl.indexOf('https://') == 0) {
1200 |             deepLink = 'music://' + webUrl.substring(8)
```



```
1201 |     } else if (webUrl.indexOf('http://') == 0) {
1202 |         deepLink = 'music://' + webUrl.substr(7)
1203 |     }
1204 |     console.log('[music-app] apple client-side hit:',
1205 |         'trackId=', trackId, 'webUrl=', webUrl)
1206 |     resolve({
1207 |         deepLink: deepLink,
1208 |         webUrl: webUrl,
1209 |         songId: trackId,
1210 |     })
1211 | } catch (_e) {
1212 |     console.warn('[music-app] apple client-side parse threw, fallback to backend')
1213 |     resolve(empty)
1214 | }
1215 | },
1216 | fail: (err) => {
1217 |     console.warn('[music-app] apple client-side iTunes request failed:', err)
1218 |     resolve(empty)
1219 | },
1220 | })
1221 | })
1222 | }
1223 | /**
1224 |  * 走后端 /api/music/resolve 的统一解析+拉起流程。
1225 |  *
1226 |  * 兜底优先级（从高到低）：
1227 |  * 1) schemeCandidates — 裸私有 scheme 逐个试（对外公开的，已注册了 BROWSABLE）
1228 |  * 2) deepLink 主链 — 跟候选独立的 Universal Link / 自定义 scheme
1229 |  * 3) setPackage + URL — 把 inAppSchemes / webUrl 锁死到目标包发 Intent,
1230 |  *   **这是"直达详情页"的关键**：setPackage 放宽 intent-filter 匹配规则
1231 |  *   （不要 BROWSABLE），能命中 App 内部未公开的 scheme 处理 Activity,
1232 |  *   或者未 autoVerify 的 App Link intent-filter
1233 |  * 4) launchAppByPackage — 按包名 launchIntent 拉起 App 首页，"必中"兜底
1234 |  * 5) webUrl — 浏览器打开，多数带"在App中打开"横幅
1235 |  * 6) fallbackOpenHomeWithClipboard — 统一兜底
1236 |  *
1237 |  * 顺序设计原则：体验越好的越靠前（直达详情 > 进App首页 > 进浏览器）。
1238 |  * 步骤 3 和 4 放在 5 前是因为 scheme 全挂时，直接把用户带进 App（即便是首页）
1239 |  * 也比让他在浏览器里再点一次"在App中打开"横幅少一步。
1240 |  *
1241 |  * 调用方负责在调用前 showLoading，本函数负责 hideLoading。
1242 |  */
1243 | function resolveAndOpenViaBackend(
1244 |     key : MusicPlatform,
1245 |     title : string,
1246 |     artist : string,
1247 |     query : string,
1248 |     name : string,
1249 | ) : void {
1250 |     runResolveMusicDeeplink(title, artist, `${key}`)
```

```
1251 | .then((resp : any) => {
1252 |   uni.hideLoading()
1253 |   // 后端返回 { ok: true, data: { deepLink, schemeCandidates?, webUrl, ... } | null }
1254 |   const data = runJoSub(resp, 'data')
1255 |   if (data == null) {
1256 |     // 后端没搜到这首歌 → 走首页 + 剪贴板兜底
1257 |     fallbackOpenHomeWithClipboard(key, query)
1258 |     return
1259 |   }
1260 |   const deepLink = runJoStr(data, 'deepLink')
1261 |   const webUrl = runJoStr(data, 'webUrl')
1262 |   const candidatesRaw = runJoArr(data, 'schemeCandidates')
1263 |   const packagesRaw = runJoArr(data, 'packageNames')
1264 |   const inAppSchemesRaw = runJoArr(data, 'inAppSchemes')
1265 |   // 酷狗 inAppSchemes 很长且每条都带内部常量 type=playone (见后端注释), 整数组刷屏易误解;
1266 |   // 只打第一条 + 总数即可。
1267 |   if (key == 'kugou' && inAppSchemesRaw != null) {
1268 |     const arr = inAppSchemesRaw as any[]
1269 |     const first = arr.length > 0 ? `${arr[0]}` : ''
1270 |     console.log(
1271 |       '[music-app] resolved:',
1272 |       'deepLink=', deepLink,
1273 |       'candidates=', candidatesRaw,
1274 |       'packages=', packagesRaw,
1275 |       'inAppSchemes count=', arr.length,
1276 |       'first=', first,
1277 |       'webUrl=', webUrl,
1278 |     )
1279 |   } else {
1280 |     console.log('[music-app] resolved:',
1281 |       'deepLink=', deepLink,
1282 |       'candidates=', candidatesRaw,
1283 |       'packages=', packagesRaw,
1284 |       'inAppSchemes=', inAppSchemesRaw,
1285 |       'webUrl=', webUrl)
1286 |   }
1287 |   // 先处理剪贴板: 需要粘贴搜索的平台 (QQ/网易/酷狗) 统一写**歌名+歌手**
1288 |   // 到剪贴板, 无论后续是跳"搜索结果页"还是"App 首页", 用户都能直接长按
1289 |   // 搜索框粘贴。
1290 |   if (needsPasteHint(key)) {
1291 |     uni.setClipboardData({
1292 |       data: query,
1293 |       success: () => {},
1294 |       fail: () => {},
1295 |     })
1296 |   }
1297 |   // 1. 优先试 schemeCandidates 列表 (按顺序)
1298 |   if (candidatesRaw != null) {
1299 |     const arr = candidatesRaw as any[]
1300 |     for (let i = 0; i < arr.length; i++) {
```

```
1301 |     const cand = `${arr[i]}`
1302 |     if (cand != "" && openDeepLink(cand)) {
1303 |         uni.showToast({ title: `已打开${name}`, icon: 'none', duration: 1500 })
1304 |         return
1305 |     }
1306 | }
1307 | }
1308 | // 2. 主深链（若不在候选里）
1309 | if (deepLink != "" && openDeepLink(deepLink)) {
1310 |     uni.showToast({ title: `已打开${name}`, icon: 'none', duration: 1500 })
1311 |     return
1312 | }
1313 | // 3. 关键步：把 inAppSchemes 通过 setPackage 发给目标 App ——
1314 | // 这是"直达歌曲详情页"的希望所在。setPackage 后 intent 匹配规则放
1315 | // 宽（不要 BROWSABLE），能命中 App 内部"未公开"的 scheme 处理 Activity
1316 | // 或"未 autoVerify 的 App Link"intent-filter。
1317 | //
1318 | // 这里**故意不把 webUrl 喂给 setPackage**：实测酷狗对自家
1319 | // m.kugou.com 的 App Link handler 不是"解析 URL 直达原生播放页"，
1320 | // 而是**把 URL 丢进 App 内置 WebView 打开**——而那个 WebView 会
1321 | // 加载失败（"网络异常"页，推测是酷狗 WebView 被自家站点反爬/需签名
1322 | // 之类）。结果反而比让 webUrl 走系统浏览器更差（浏览器里 m.kugou.com
1323 | // 能正常加载 + 带"在App中打开"横幅）。所以 webUrl 统一下放到 step 5。
1324 | // `inAppSchemes` 只装"私有 scheme"（形如 kugouURL://start.weixin?...），
1325 | // 这些走的是 App 内部的 Activity 路由而非 WebView，安全。
1326 | console.log(`[music-app] step 3: tryOpenInPackages (inAppSchemes + setPackage)`)
1327 | const urlsInPackage : string[] = []
1328 | if (inAppSchemesRaw != null) {
1329 |     const arr = inAppSchemesRaw as any[]
1330 |     for (let i = 0; i < arr.length; i++) {
1331 |         const s = `${arr[i]}`
1332 |         if (s != "") urlsInPackage.push(s)
1333 |     }
1334 | }
1335 | if (tryOpenInPackages(urlsInPackage, packagesRaw)) {
1336 |     // 酷狗直达详情页成功：首次跳转解释"未知歌手"问题，后续只 toast
1337 |     if (key == 'kugou') {
1338 |         showKugouMetaHintOnce()
1339 |     } else {
1340 |         uni.showToast({ title: `已打开${name}`, icon: 'none', duration: 1500 })
1341 |     }
1342 |     return
1343 | }
1344 | // 4. Android: 按包名拉起 App —— scheme 全挂也能点一下进 App 主页。
1345 | // 剪贴板已写入歌名+歌手，toast 提示用户到搜索框粘贴。
1346 | console.log(`[music-app] step 4: tryLaunchByPackages`)
1347 | if (tryLaunchByPackages(packagesRaw)) {
1348 |     uni.showToast({
1349 |         title: needsPasteHint(key)
1350 |         ? `已打开${name}，到搜索框粘贴即可播放`
```

```
1351 |       : `已打开${name}`,
1352 |       icon: 'none',
1353 |       duration: 2500,
1354 |     })
1355 |     return
1356 |   }
1357 |   // 5. webUrl (Universal Link / 移动版网页, 浏览器打开)
1358 |   console.log('[music-app] step 5: openDeepLink(webUrl)')
1359 |   if (webUrl !== '' && openDeepLink(webUrl)) {
1360 |     uni.showToast({
1361 |       title: needsPasteHint(key) ? `已打开${name}网页, 按提示进入App` : `已打开${name}`,
1362 |       icon: 'none',
1363 |       duration: 2000,
1364 |     })
1365 |     return
1366 |   }
1367 |   // 走到这: 一切都挂了 → 退到通用兜底
1368 |   fallbackOpenHomeWithClipboard(key, query)
1369 | })
1370 | .catch((err: any | null) => {
1371 |   uni.hideLoading()
1372 |   console.warn('[openSongInMusicApp] resolve failed, fallback:', err)
1373 |   fallbackOpenHomeWithClipboard(key, query)
1374 | })
1375 | }
1376 | /**
1377 |  * 推荐入口: 带上歌名+歌手, 先尝试拉起对应音乐 App 直达单曲页。
1378 |  *
1379 |  * 流程:
1380 |  * - 小程序端: 不能拉外部 App, 直接走兜底弹窗 (fallbackOpenHomeWithClipboard)
1381 |  * - Apple Music 平台: 两段 iTunes Search 双保险拿 trackId:
1382 |  *   1) 客户端直连 itunes.apple.com (resolveAppleClientSide) ——
1383 |  *     国内 iPhone 运营商网络通常能直达, 命中 ≈ 90% case
1384 |  *   2) 客户端 miss → 后端 /api/music/resolve 再试一次 ——
1385 |  *     后端部署到海外节点能稳定直连
1386 |  *   **两段都拿不到 trackId 时不再拉 Apple Music** (避免"拉起 App 停首页"的
1387 |  *     不诚实体验, 详见 appleNoMatchToast 注释), 只 toast + 复制歌名
1388 |  * - 其它平台: 直接走后端 /api/music/resolve (resolveAndOpenViaBackend
1389 |  *   里面有 schemeCandidates / setPackage / 网页兜底等多级降级)
1390 |  *
1391 |  * 为什么 Apple 走客户端先: dev 后端在国内直连 itunes.apple.com 经常 ECONNRESET,
1392 |  * 但用户终端通常通; 这是用户那一侧拿到「真·单曲页深链」的唯一稳定路径。
1393 |  * 详细原因见 resolveAppleClientSide 注释。
1394 |  */
1395 | export function openSongInMusicApp(
1396 |   key: MusicPlatform,
1397 |   title: string,
1398 |   artist: string,
1399 | ): void {
1400 |   const query = buildMusicQuery(title, artist)
```

```
1401 | if (query == '') return
1402 | const name = platformLabel(key)
1403 | // #ifdef MP-WEIXIN || MP-ALIPAY || MP-BAIDU || MP-TOUTIAO || MP-QQ || MP-KUAISHOU || MP-JD...
1404 | // 小程序不能拉外部 App，直接弹提示 + 复制
1405 | fallbackOpenHomeWithClipboard(key, query)
1406 | return
1407 | // #endif
1408 | // #ifndef MP-WEIXIN || MP-ALIPAY || MP-BAIDU || MP-TOUTIAO || MP-QQ || MP-KUAISHOU || MP-J...
1409 | // 先给一个 loading，网络慢也让用户知道点击有反应
1410 | uni.showLoading({ title: `正在打开${(name)}...`, mask: true })
1411 | if (key == 'apple') {
1412 |   // Apple 专用流程，跟其它平台不同点（关键，否则会"拉起 App 但停首页"）：
1413 |   // - 走两段 iTunes Search：先客户端直连(resolveAppleClientSide)，miss 后退
1414 |   //   后端(/api/music/resolve)，**任一段命中**就用真 trackId 拼 music://
1415 |   //   单曲深链直跳详情页；
1416 |   // - **两段都拿不到 trackId 时不再拉 Apple Music**（后端会兜底返回
1417 |   //   search Universal Link 但 songId="" —— iOS 实测这种 search URL 进 App
1418 |   //   只能停首页，term 不被解析），改走 appleNoMatchToast: toast + 复制
1419 |   //   歌名，不打开 App。
1420 |   //
1421 |   // 后端 search fallback 的识别信号：data.songId === ''（见 backend
1422 |   //   appleSearchFallback()）。
1423 |   resolveAppleClientSide(title, artist)
1424 |   .then((appleRes : any) => {
1425 |     // resolveAppleClientSide 永远 resolve 同一种 shape，**不会** resolve null
1426 |     //（Android Kotlin 编译禁不能给 Promise<Any> 传 null，详见函数顶部注释）。
1427 |     // 用 songId 是否为空判 miss，与后端口径完全一致。
1428 |     const clientSongId = runJoStr(appleRes, 'songId')
1429 |     if (clientSongId != '') {
1430 |       // 客户端命中：先用 music:// 直跳 App 单曲页；不行再用 https
1431 |       // Universal Link（装了 App 走 applinks，没装走 Safari）。
1432 |       // 字段读取统一走 runJoStr：原生端 resolve 值会变成 UTSJSONObject，
1433 |       // 不能直接`.deepLink`属性访问。
1434 |       uni.hideLoading()
1435 |       const dl = runJoStr(appleRes, 'deepLink')
1436 |       const wu = runJoStr(appleRes, 'webUrl')
1437 |       if (dl != '' && openDeepLink(dl)) {
1438 |         uni.showToast({ title: `已打开${(name)}`, icon: 'none', duration: 1500 })
1439 |         return
1440 |       }
1441 |       if (wu != '' && openDeepLink(wu)) {
1442 |         uni.showToast({ title: `已打开${(name)}`, icon: 'none', duration: 1500 })
1443 |         return
1444 |       }
1445 |       // music:// 和 https 都拒绝（极罕见，几乎只发生在 iOS 连 Safari 都拒绝
1446 |       // 处理 https://music.apple.com 时）→ 走"未找到"兜底
1447 |       appleNoMatchToast(query)
1448 |       return
1449 |     }
1450 |     // 客户端 miss → 转后端再试一次（后端在阿里云国际/海外节点能直连
```

```
1451 | // iTunes, dev 在国内常 ECONNRESET 但生产环境正常)
1452 | runResolveMusicDeeplink(title, artist, 'apple')
1453 | .then((resp : any) => {
1454 |     uni.hideLoading()
1455 |     const data = runJoSub(resp, 'data')
1456 |     if (data == null) {
1457 |         // 后端 ok=false 或 data=null: iTunes 完全没响应
1458 |         appleNoMatchToast(query)
1459 |         return
1460 |     }
1461 |     // 关键判别: songId 非空 = 后端 iTunes 真命中; songId 为空 = 后端
1462 |     // 也只能给 search universal link 兜底, 对 Apple 来说等于没用
1463 |     // (详见 appleNoMatchToast 顶部注释)
1464 |     const songId = runJoStr(data, 'songId')
1465 |     if (songId == '') {
1466 |         console.log('[music-app] apple backend: search-fallback only (no trackId), skip...
1467 |         appleNoMatchToast(query)
1468 |         return
1469 |     }
1470 |     const dl = runJoStr(data, 'deepLink')
1471 |     const wu = runJoStr(data, 'webUrl')
1472 |     console.log('[music-app] apple backend hit: trackId=', songId, 'webUrl=', wu)
1473 |     if (dl != '' && openDeepLink(dl)) {
1474 |         uni.showToast({ title: `已打开${name}`, icon: 'none', duration: 1500 })
1475 |         return
1476 |     }
1477 |     if (wu != '' && openDeepLink(wu)) {
1478 |         uni.showToast({ title: `已打开${name}`, icon: 'none', duration: 1500 })
1479 |         return
1480 |     }
1481 |     appleNoMatchToast(query)
1482 | })
1483 | .catch((err : any | null) => {
1484 |     uni.hideLoading()
1485 |     console.warn('[music-app] apple backend resolve failed:', err)
1486 |     appleNoMatchToast(query)
1487 | })
1488 | })
1489 | .catch((_err : any | null) => {
1490 |     // resolveAppleClientSide 内部已经把异常转成 miss-shape resolve 了, 这里
1491 |     // 几乎走不到; 真出意外也别让 loading 卡死, 直接给"未找到"兜底
1492 |     uni.hideLoading()
1493 |     appleNoMatchToast(query)
1494 | })
1495 | return
1496 | }
1497 | // 非 Apple 平台: 直接走后端
1498 | resolveAndOpenViaBackend(key, title, artist, query, name)
1499 | // #endif
1500 | }
```

```
1501 | /**
1502 |  * 老入口: 只拉起 App 首页 (或搜索页) + 复制歌名到剪贴板。
1503 |  * 不需要直达歌曲播放页时可用; 新代码请优先用 openSongInMusicApp。
1504 |  */
1505 | export function openInMusicApp(key : MusicPlatform, query : string) : void {
1506 |   fallbackOpenHomeWithClipboard(key, query)
1507 | }
1508 | // ===== utils/number.uts =====
1509 | /** UTS 下避免 Number.isFinite / Infinity 字面量差异, 用范围判断有限数 */
1510 | export function isFiniteNumber(n : number) : boolean {
1511 |   if (n !== n) {
1512 |     return false
1513 |   }
1514 |   const a = n > 0 ? n : 0 - n
1515 |   return a <= 1.7976931348623157e308
1516 | }
1517 | // ===== utils/run-uts.uts =====
1518 | /**
1519 |  * 读接口 JSON 字段 (避免对 any 使用 .items / .tracks 等点号, UTS error18) 。
1520 |  * 原生端用 UTSJSONObject.getString/getAny/getJSON, H5 端 res.data 是 plain Object,
1521 |  * .getString 等并不存在 (报 "jo.getString is not a function") , 所以按平台条件编译。
1522 |  */
1523 | import { isFiniteNumber } from '@utils/number.uts'
1524 | /** 统一读字符串: 任一端都先确保非空 */
1525 | export function runJoStr(el : any, k : string) : string {
1526 |   if (el == null) return ""
1527 |   // #ifdef H5
1528 |   const v = (el as any)[k]
1529 |   return v !== null ? `${v}` : ""
1530 |   // #endif
1531 |   // #ifndef H5
1532 |   const jo = el as UTSJSONObject
1533 |   const s = jo.getString(k)
1534 |   return s !== null ? s : ""
1535 |   // #endif
1536 | }
1537 | /** 读可能为空的整数: null 保留, 非法数字转 null */
1538 | export function runJoOptInt(el : any, k : string) : number | null {
1539 |   if (el == null) return null
1540 |   // #ifdef H5
1541 |   const a = (el as any)[k]
1542 |   if (a == null) return null
1543 |   const n = parseInt(`${a}`, 10)
1544 |   return isFiniteNumber(n) ? n : null
1545 |   // #endif
1546 |   // #ifndef H5
1547 |   const jo = el as UTSJSONObject
1548 |   const a2 = jo.getAny(k)
1549 |   if (a2 == null) return null
1550 |   const n2 = parseInt(`${a2}`, 10)
```

```
1551 | return isFiniteNumber(n2) ? n2 : null
1552 | // #endif
1553 | }
1554 | export function runJoInt(el : any, k : string, def : number) : number {
1555 |   const n = runJoOptInt(el, k)
1556 |   return n != null ? n : def
1557 | }
1558 | /** 读嵌套对象：原生返回 UTSJSONObject，H5 返回 plain Object（仍用 any 往下传） */
1559 | export function runJoSub(el : any, k : string) : any | null {
1560 |   if (el == null) return null
1561 |   // #ifdef H5
1562 |   return (el as any)[k]
1563 |   // #endif
1564 |   // #ifndef H5
1565 |   const jo = el as UTSJSONObject
1566 |   return jo.getJSON(k)
1567 |   // #endif
1568 | }
1569 | /** 读数组（给列表类字段用） */
1570 | export function runJoArr(el : any, k : string) : any | null {
1571 |   if (el == null) return null
1572 |   // #ifdef H5
1573 |   return (el as any)[k]
1574 |   // #endif
1575 |   // #ifndef H5
1576 |   const jo = el as UTSJSONObject
1577 |   return jo.getAny(k)
1578 |   // #endif
1579 | }
1580 | /** 真机接口数组元素多为 UTSJSONObject，v-for 不能直接 as 业务类型，需逐项解析 */
1581 | export type UtsHotTrackItem = {
1582 |   id?: number
1583 |   title: string
1584 |   artist: string
1585 |   bpm: number
1586 |   motionForm?: string
1587 | }
1588 | export function utsNormalizeHotTrackList(raw : any | null) : UtsHotTrackItem[] {
1589 |   if (raw == null) {
1590 |     return []
1591 |   }
1592 |   const arr = raw as any[]
1593 |   const out : UtsHotTrackItem[] = []
1594 |   for (let i = 0; i < arr.length; i++) {
1595 |     const el = arr[i]
1596 |     const row : UtsHotTrackItem = {
1597 |       title: runJoStr(el, 'title'),
1598 |       artist: runJoStr(el, 'artist'),
1599 |       bpm: runJoInt(el, 'bpm', 0),
1600 |     }
```



```
1601 |   const idn = runJoOptInt(el, 'id')
1602 |   if (idn !== null) {
1603 |     row.id = idn
1604 |   }
1605 |   const mf = runJoStr(el, 'motionForm')
1606 |   if (mf !== '') {
1607 |     row.motionForm = mf
1608 |   }
1609 |   out.push(row)
1610 | }
1611 | return out
1612 |}
1613 | export type UtsOnboardSpeedCompare = {
1614 |   label: string
1615 |}
1616 | export type UtsOnboardTrackRow = {
1617 |   id?: number
1618 |   title: string
1619 |   artist: string
1620 |   bpm: number
1621 |   hint?: string
1622 |   speedCompare?: UtsOnboardSpeedCompare
1623 |}
1624 | export function utsNormalizeOnboardTrackList(raw : any | null) : UtsOnboardTrackRow[] {
1625 |   if (raw === null) {
1626 |     return []
1627 |   }
1628 |   const arr = raw as any[]
1629 |   const out : UtsOnboardTrackRow[] = []
1630 |   for (let i = 0; i < arr.length; i++) {
1631 |     const el = arr[i]
1632 |     const row : UtsOnboardTrackRow = {
1633 |       title: runJoStr(el, 'title'),
1634 |       artist: runJoStr(el, 'artist'),
1635 |       bpm: runJoInt(el, 'bpm', 0),
1636 |     }
1637 |     const idn = runJoOptInt(el, 'id')
1638 |     if (idn !== null) {
1639 |       row.id = idn
1640 |     }
1641 |     const h = runJoStr(el, 'hint')
1642 |     if (h !== '') {
1643 |       row.hint = h
1644 |     }
1645 |     const scJo = runJoSub(el, 'speedCompare')
1646 |     if (scJo !== null) {
1647 |       const lab = runJoStr(scJo, 'label')
1648 |       if (lab !== '') {
1649 |         const sc : UtsOnboardSpeedCompare = { label: lab }
1650 |         row.speedCompare = sc
```

```
1651 |   }
1652 | }
1653 |   out.push(row)
1654 | }
1655 | return out
1656 | }
1657 | export type UtsRunTrackRow = {
1658 |   id?: number
1659 |   title: string
1660 |   artist: string
1661 |   bpm: number
1662 | }
1663 | export function utsNormalizeRunTrackRowList(raw : any | null) : UtsRunTrackRow[] {
1664 |   if (raw == null) {
1665 |     return []
1666 |   }
1667 |   const arr = raw as any[]
1668 |   const out : UtsRunTrackRow[] = []
1669 |   for (let i = 0; i < arr.length; i++) {
1670 |     const el = arr[i]
1671 |     const row : UtsRunTrackRow = {
1672 |       title: runJoStr(el, 'title'),
1673 |       artist: runJoStr(el, 'artist'),
1674 |       bpm: runJoInt(el, 'bpm', 0),
1675 |     }
1676 |     const idn = runJoOptInt(el, 'id')
1677 |     if (idn != null) {
1678 |       row.id = idn
1679 |     }
1680 |     out.push(row)
1681 |   }
1682 |   return out
1683 | }
1684 | export type UtsSearchSpeedCompare = {
1685 |   kind: string
1686 |   label: string
1687 | }
1688 | export type UtsSearchTrackItem = {
1689 |   id?: number
1690 |   title: string
1691 |   artist: string
1692 |   bpm: number
1693 |   motionForm?: string
1694 |   language?: string
1695 |   speedCompare?: UtsSearchSpeedCompare
1696 | }
1697 | export function utsNormalizeSearchTrackList(raw : any | null) : UtsSearchTrackItem[] {
1698 |   if (raw == null) {
1699 |     return []
1700 |   }
```

```
1701 | const arr = raw as any[]
1702 | const out : UtsSearchTrackItem[] = []
1703 | for (let i = 0; i < arr.length; i++) {
1704 |     const el = arr[i]
1705 |     const row : UtsSearchTrackItem = {
1706 |         title: runJoStr(el, 'title'),
1707 |         artist: runJoStr(el, 'artist'),
1708 |         bpm: runJoInt(el, 'bpm', 0),
1709 |     }
1710 |     const idn = runJoOptInt(el, 'id')
1711 |     if (idn != null) {
1712 |         row.id = idn
1713 |     }
1714 |     const mf = runJoStr(el, 'motionForm')
1715 |     if (mf != '') row.motionForm = mf
1716 |     const lang = runJoStr(el, 'language')
1717 |     if (lang != '') row.language = lang
1718 |     const scJo = runJoSub(el, 'speedCompare')
1719 |     if (scJo != null) {
1720 |         const k = runJoStr(scJo, 'kind')
1721 |         const l = runJoStr(scJo, 'label')
1722 |         row.speedCompare = {
1723 |             kind: k,
1724 |             label: l,
1725 |         }
1726 |     }
1727 |     out.push(row)
1728 | }
1729 | return out
1730 | }
1731 | export type UtsShareTpl = {
1732 |     title: string
1733 |     description?: string
1734 |     motionForm: string
1735 |     speedFeel: string
1736 |     bpmMin: number
1737 |     bpmMax: number
1738 |     refTrackTitle?: string
1739 |     refTrackArtist?: string
1740 | }
1741 | export function utsNormalizeShareTpl(raw : any | null) : UtsShareTpl | null {
1742 |     if (raw == null) {
1743 |         return null
1744 |     }
1745 |     const el = raw
1746 |     const row : UtsShareTpl = {
1747 |         title: runJoStr(el, 'title'),
1748 |         motionForm: runJoStr(el, 'motionForm'),
1749 |         speedFeel: runJoStr(el, 'speedFeel'),
1750 |         bpmMin: runJoInt(el, 'bpmMin', 0),
```

```
1751 |   bpmMax: runJoInt(el, 'bpmMax', 0),
1752 | }
1753 | const desc = runJoStr(el, 'description')
1754 | if (desc !== '') {
1755 |   row.description = desc
1756 | }
1757 | const rt = runJoStr(el, 'refTrackTitle')
1758 | if (rt !== '') {
1759 |   row.refTrackTitle = rt
1760 | }
1761 | const ra = runJoStr(el, 'refTrackArtist')
1762 | if (ra !== '') {
1763 |   row.refTrackArtist = ra
1764 | }
1765 | return row
1766 |}
1767 | export type UtsRecentTplRow = {
1768 |   title: string
1769 |   bpmMin: number
1770 |   bpmMax: number
1771 |   motionForm: string
1772 |   speedFeel: string
1773 |   shareCode?: string
1774 |}
1775 | export function utsNormalizeRecentTplList(raw : any | null) : UtsRecentTplRow[] {
1776 |   if (raw == null) {
1777 |     return []
1778 |   }
1779 |   const arr = raw as any[]
1780 |   const out : UtsRecentTplRow[] = []
1781 |   for (let i = 0; i < arr.length; i++) {
1782 |     const el = arr[i]
1783 |     const row : UtsRecentTplRow = {
1784 |       title: runJoStr(el, 'title'),
1785 |       bpmMin: runJoInt(el, 'bpmMin', 0),
1786 |       bpmMax: runJoInt(el, 'bpmMax', 0),
1787 |       motionForm: runJoStr(el, 'motionForm'),
1788 |       speedFeel: runJoStr(el, 'speedFeel'),
1789 |     }
1790 |     const sc = runJoStr(el, 'shareCode')
1791 |     if (sc !== '') {
1792 |       row.shareCode = sc
1793 |     }
1794 |     out.push(row)
1795 |   }
1796 |   return out
1797 |}
1798 | export type UtsPlaylistListRow = {
1799 |   id?: number
1800 |   name: string
```

```
1801 | trackCount: number
1802 | description?: string
1803 | motionForm?: string
1804 | }
1805 | export function utsNormalizePlaylistListRows(raw : any | null) : UtsPlaylistListRow[] {
1806 |   if (raw == null) {
1807 |     return []
1808 |   }
1809 |   const arr = raw as any[]
1810 |   const out : UtsPlaylistListRow[] = []
1811 |   for (let i = 0; i < arr.length; i++) {
1812 |     const el = arr[i]
1813 |     const row : UtsPlaylistListRow = {
1814 |       name: runJoStr(el, 'name'),
1815 |       trackCount: runJoInt(el, 'trackCount', 0),
1816 |     }
1817 |     const idn = runJoOptInt(el, 'id')
1818 |     if (idn != null) {
1819 |       row.id = idn
1820 |     }
1821 |     const desc = runJoStr(el, 'description')
1822 |     if (desc != '') {
1823 |       row.description = desc
1824 |     }
1825 |     const mf = runJoStr(el, 'motionForm')
1826 |     if (mf != '') {
1827 |       row.motionForm = mf
1828 |     }
1829 |     out.push(row)
1830 |   }
1831 |   return out
1832 | }
1833 | export type UtsPlaylistDetailModel = {
1834 |   name: string
1835 |   description?: string
1836 |   motionForm?: string
1837 |   tracks: UtsRunTrackRow[]
1838 | }
1839 | export function utsNormalizePlaylistDetail(raw : any | null) : UtsPlaylistDetailModel | null {
1840 |   if (raw == null) {
1841 |     return null
1842 |   }
1843 |   const root = raw
1844 |   const d : UtsPlaylistDetailModel = {
1845 |     name: runJoStr(root, 'name'),
1846 |     tracks: utsNormalizeRunTrackRowList(runJoArr(root, 'tracks')),
1847 |   }
1848 |   const desc = runJoStr(root, 'description')
1849 |   if (desc != '') {
1850 |     d.description = desc
```

```
1851 | }
1852 | const mf = runJoStr(root, 'motionForm')
1853 | if (mf !== '') {
1854 |   d.motionForm = mf
1855 | }
1856 | return d
1857 |}
1858 | export type UtsSongDetailTrack = {
1859 |   title: string
1860 |   artist: string
1861 |   bpm: number
1862 |   genre?: string
1863 |   language?: string
1864 |}
1865 | export function utsNormalizeSongDetailTrack(raw : any | null) : UtsSongDetailTrack | null {
1866 |   if (raw == null) {
1867 |     return null
1868 |   }
1869 |   const el = raw
1870 |   const row : UtsSongDetailTrack = {
1871 |     title: runJoStr(el, 'title'),
1872 |     artist: runJoStr(el, 'artist'),
1873 |     bpm: runJoinInt(el, 'bpm', 0),
1874 |   }
1875 |   const g = runJoStr(el, 'genre')
1876 |   if (g !== '') {
1877 |     row.genre = g
1878 |   }
1879 |   const lang = runJoStr(el, 'language')
1880 |   if (lang !== '') {
1881 |     row.language = lang
1882 |   }
1883 |   return row
1884 |}
1885 | /**
1886 |  * 应用内打开页面: 优先 navigateTo 保留栈; 失败时用 reLaunch 兜底 (会清栈, 是已知权衡)。
1887 |  *
1888 |  * 这里加一层"进行中"锁代替时间戳防抖:
1889 |  * - uni-app-x 的 navigateTo 有自己的路由锁, 动画未结束时二次调用会报
1890 |  *   errCode=4 + message "/pages/xxx locked"。
1891 |  * - 即使老代码里 reLaunch 能兜底打开页面, 多次触发会导致日志刷屏、偶发卡死 (见
1892 |  *   DCloud 论坛 question/200896) 。
1893 |  * - 用一个 _busy 标记, 配合 success / fail 回调 + 一点兜底定时器解锁, 保证同一时刻
1894 |  *   只有一次跳转在飞。
1895 |  */
1896 | let _navBusy : boolean = false
1897 | let _navBusyUrl : string = ''
1898 | function releaseNavBusy() {
1899 |   _navBusy = false
1900 |   _navBusyUrl = ''
```

```
1901 | }
1902 | export function runOpenPage(url : string) : void {
1903 |   // 同 URL 正在跳就丢弃；不同 URL 但也还在跳，也丢弃（防止用户连点两项不同的歌）
1904 |   if (_navBusy) {
1905 |     console.log('[runOpenPage] navigation busy, ignored:', url, 'busyUrl=', _navBusyUrl)
1906 |     return
1907 |   }
1908 |   _navBusy = true
1909 |   _navBusyUrl = url
1910 |   // 双保险：无论 success / fail 有没有触发，600ms 后强制解锁，避免锁死
1911 |   // 页面动画 260ms，加 340ms 余量
1912 |   setTimeout(() => { releaseNavBusy() }, 600)
1913 |   uni.navigateTo({
1914 |     url: url,
1915 |     success: () => {
1916 |       releaseNavBusy()
1917 |     },
1918 |     fail: (err) => {
1919 |       console.warn('[runOpenPage] navigateTo failed', url, err)
1920 |       // reLaunch 兜底：保持"点了就能打开"的行为（注意会清空页面栈）
1921 |       uni.reLaunch({
1922 |         url: url,
1923 |         success: () => {
1924 |           releaseNavBusy()
1925 |         },
1926 |         fail: (err2) => {
1927 |           releaseNavBusy()
1928 |           console.error('[runOpenPage] reLaunch failed', url, err2)
1929 |           uni.showToast({
1930 |             title: `打开失败: ${url}`,
1931 |             icon: 'none',
1932 |             duration: 2500,
1933 |           })
1934 |         },
1935 |       })
1936 |     },
1937 |   })
1938 | }
1939 | export function utsGetAny(o : any, key : string) : any | null {
1940 |   if (o == null) return null
1941 |   // #ifdef H5
1942 |   return (o as any)[key]
1943 |   // #endif
1944 |   // #ifndef H5
1945 |   const jo = o as UTSJSONObject
1946 |   return jo.getAny(key)
1947 |   // #endif
1948 | }
1949 | export function utsGetStringOrEmpty(o : any, key : string) : string {
1950 |   return runJoStr(o, key)
```

```
1951 | }
1952 | /**
1953 |  * 本地 BPM 分类, 规则与后端 `domain/motion/BpmClassification.ts` 一致。
1954 |  * 放到前端省掉 song-detail 额外一次 /bpm/classify 往返。
1955 |  * 返回的是"已经去掉 markdown 星号"的人类可读摘要。
1956 |  */
1957 | export function utsClassifyBpmSummary(bpm : number) : string {
1958 |   if (!isFiniteNumber(bpm) || bpm <= 0) return ""
1959 |   let n = Math.round(bpm)
1960 |   if (n < 40) n = 40
1961 |   if (n > 220) n = 220
1962 |   let isWalk : boolean = n <= 120
1963 |   let speedLabel : string = ""
1964 |   if (isWalk) {
1965 |     if (n < 100) speedLabel = '较慢'
1966 |     else if (n <= 112) speedLabel = '中等'
1967 |     else speedLabel = '快'
1968 |   } else {
1969 |     if (n < 135) speedLabel = '较慢'
1970 |     else if (n <= 155) speedLabel = '中等'
1971 |     else speedLabel = '快'
1972 |   }
1973 |   const verb = isWalk ? '快走/散步' : '慢跑/节奏跑'
1974 |   return `这首歌适合${verb}, 速度${speedLabel} (BPM: ${n})`
1975 | }
1976 | /** 引导参照曲条目: onboarding 试听用 */
1977 | export type UtsOnboardingRefTrack = {
1978 |   id: number
1979 |   title: string
1980 |   artist: string
1981 |   bpm: number
1982 |   motionForm: string
1983 |   audioUrl: string
1984 | }
1985 | export function utsNormalizeOnboardingRefTrack(raw : any | null) : UtsOnboardingRefTrack | nu...
1986 | if (raw == null) return null
1987 | const el = raw
1988 | const idn = runJoOptInt(el, 'id')
1989 | const bpm = runJoInt(el, 'bpm', 0)
1990 | const url = runJoStr(el, 'audioUrl')
1991 | if (idn == null || bpm <= 0 || url == "") return null
1992 | return {
1993 |   id: idn,
1994 |   title: runJoStr(el, 'title'),
1995 |   artist: runJoStr(el, 'artist'),
1996 |   bpm: bpm,
1997 |   motionForm: runJoStr(el, 'motionForm'),
1998 |   audioUrl: url,
1999 | }
2000 | }
```



```
2001 | /** picker @change 的 e.detail.value (勿写 e.detail, UTS 会解析失败) */
2002 | export function utsPickerChangeIndex(e : any) : number {
2003 |   if (e == null) return 0
2004 |   const det = runJoSub(e, 'detail')
2005 |   if (det == null) return 0
2006 |   const v = utsGetAny(det, 'value')
2007 |   return parseInt(`${v}`, 10)
2008 | }
2009 | // ===== pages/splash/splash.uvue =====
2010 | <template>
2011 |   <view class="splash-container" :class="{ 'theme-dark': isDarkMode }">
2012 |     <text class="app-name">行境</text>
2013 |     <text class="app-tag">专属音乐运动节拍器, 建议佩戴耳机</text>
2014 |     <image class="loading-image" src="/static/loading.png" mode="aspectFit"> </image>
2015 |     <text class="loading-text">正在启动</text>
2016 |     <view class="progress-container">
2017 |       <view class="progress-bar" :style="{ width: progress + '%' }"> </view>
2018 |     </view>
2019 |     <text class="progress-text">{{ progress }}%</text>
2020 |   </view>
2021 | </template>
2022 | <script lang="uts">
2023 |   import { state } from '@store/index.uts'
2024 |   export default {
2025 |     data() {
2026 |       return {
2027 |         progress: 0 as number,
2028 |       },
2029 |     },
2030 |     computed: {
2031 |       isDarkMode() : boolean {
2032 |         return state.isDarkMode
2033 |       },
2034 |     },
2035 |     onLoad() {
2036 |       this.startLoading()
2037 |     },
2038 |     methods: {
2039 |       startLoading() {
2040 |         // 100 / 4 = 25 个 tick * 100ms = 2500ms 走完进度条; +300ms 停留 = 总 ~2.8s
2041 |         // 给用户读完那句广告语和"建议佩戴耳机"的时间
2042 |         let tick = 0 as number
2043 |         tick = setInterval(() => {
2044 |           this.progress += 4
2045 |           if (this.progress >= 100) {
2046 |             this.progress = 100
2047 |             clearInterval(tick)
2048 |             setTimeout(() => {
2049 |               uni.reLaunch({
2050 |                 url: '/pages/run/index'
```

```
2051 |     })
2052 |     }, 300)
2053 |   }
2054 |   }, 100) as number
2055 | }
2056 | }
2057 | }
2058 | </script>
2059 | <style>
2060 | .splash-container {
2061 |   flex: 1;
2062 |   display: flex;
2063 |   flex-direction: column;
2064 |   align-items: center;
2065 |   justify-content: center;
2066 |   background-color: #0f1419;
2067 |   padding: 40px;
2068 | }
2069 | .splash-container.theme-dark {
2070 |   background-color: #0a0d12;
2071 | }
2072 | .app-name {
2073 |   font-size: 22px;
2074 |   font-weight: 700;
2075 |   color: #f2f4f8;
2076 |   margin-bottom: 8px;
2077 | }
2078 | .app-tag {
2079 |   font-size: 14px;
2080 |   color: #8b98a5;
2081 |   margin-bottom: 28px;
2082 | }
2083 | .loading-image {
2084 |   width: 160px;
2085 |   height: 160px;
2086 |   margin-bottom: 28px;
2087 | }
2088 | .loading-text {
2089 |   font-size: 16px;
2090 |   color: #e7ecf3;
2091 |   margin-bottom: 24px;
2092 | }
2093 | .splash-container.theme-dark .loading-text {
2094 |   color: #e7ecf3;
2095 | }
2096 | .progress-container {
2097 |   width: 280px;
2098 |   height: 6px;
2099 |   background-color: #1c2430;
2100 |   border-radius: 3px;
```

```
2101 | overflow: hidden;
2102 | margin-bottom: 16px;
2103 | }
2104 | .splash-container.theme-dark .progress-container {
2105 |   background-color: #1c2430;
2106 | }
2107 | .progress-bar {
2108 |   height: 100%;
2109 |   background-color: #38bdf8;
2110 |   border-radius: 3px;
2111 |   transition: width 0.08s ease;
2112 | }
2113 | .progress-text {
2114 |   font-size: 13px;
2115 |   color: #8b98a5;
2116 | }
2117 | .splash-container.theme-dark .progress-text {
2118 |   color: #8b98a5;
2119 | }
2120 | </style>
2121 | // ===== pages/auth/forgot-password.uvue =====
2122 | <template>
2123 |   <view class="container">
2124 |     <view class="box">
2125 |       <view class="logo-section">
2126 |         <text class="logo-text">找回密码</text>
2127 |         <text class="subtitle">输入注册手机号, 用短信验证码重置密码</text>
2128 |       </view>
2129 |       <view class="form-section">
2130 |         <view class="input-group">
2131 |           <text class="label">手机号</text>
2132 |           <input
2133 |             class="input"
2134 |             type="number"
2135 |             maxlength="11"
2136 |             placeholder="请输入注册时的手机号"
2137 |             :value="phone"
2138 |             @input="onPhoneInput"
2139 |           />
2140 |         </view>
2141 |         <view class="input-group">
2142 |           <text class="label">短信验证码</text>
2143 |           <view class="sms-row">
2144 |             <input
2145 |               class="input sms-input"
2146 |               type="number"
2147 |               maxlength="6"
2148 |               placeholder="6 位验证码"
2149 |               :value="smsCode"
2150 |               @input="onSmsCodeInput"
```

```
2151 |     />
2152 |     <button
2153 |         class="sms-btn"
2154 |         :class="{ 'sms-btn-disabled': smsCooldown > 0 || smsSending }"
2155 |         @click="handleSendSms"
2156 |         :disabled="smsCooldown > 0 || smsSending"
2157 |     >{{ smsBtnLabel }}</button>
2158 | </view>
2159 | </view>
2160 | <view class="input-group">
2161 |     <view class="label-row">
2162 |         <text class="label">新密码</text>
2163 |         <text class="eye" hover-class="eye-hover" hover-stay-time="70" @click="toggleShow..."
2164 |     </view>
2165 |     <input
2166 |         class="input"
2167 |         :type="showPw ? 'text' : 'password'"
2168 |         placeholder="6-32 位"
2169 |         :value="newPassword"
2170 |         @input="onNewPwInput"
2171 |     />
2172 | </view>
2173 | <view class="input-group">
2174 |     <view class="label-row">
2175 |         <text class="label">确认新密码</text>
2176 |         <text class="eye" hover-class="eye-hover" hover-stay-time="70" @click="toggleShow..."
2177 |     </view>
2178 |     <input
2179 |         class="input"
2180 |         :type="showPw2 ? 'text' : 'password'"
2181 |         placeholder="请再次输入新密码"
2182 |         :value="confirmPassword"
2183 |         @input="onConfirmInput"
2184 |     />
2185 | </view>
2186 | <button
2187 |     class="primary-btn"
2188 |     @click="handleReset"
2189 |     :disabled="loading"
2190 | >{{ loading ? '重置中...' : '重置密码' }}</button>
2191 | <view class="footer-links">
2192 |     <text class="link-text" @click="goBack">返回登录</text>
2193 | </view>
2194 | </view>
2195 | </view>
2196 | </view>
2197 | </template>
2198 | <script lang="uts">
2199 | import {
2200 |     authResetPassword,
```

```
2201 |   authSendSmsCode,
2202 |   buildResetPasswordPayload,
2203 |   buildSendSmsPayload,
2204 | } from '@utils/api'
2205 | import {
2206 |   utsAuthReadDevCode,
2207 |   friendlyAuthError,
2208 | } from '@utils/auth-uts.uts'
2209 | export default {
2210 |   data() {
2211 |     return {
2212 |       phone: '' as string,
2213 |       smsCode: '' as string,
2214 |       newPassword: '' as string,
2215 |       confirmPassword: '' as string,
2216 |       loading: false as boolean,
2217 |       showPw: false as boolean,
2218 |       showPw2: false as boolean,
2219 |       smsCooldown: 0 as number,
2220 |       smsSending: false as boolean,
2221 |       cooldownTimer: -1 as number,
2222 |     }
2223 |   },
2224 |   computed: {
2225 |     smsBtnLabel(): string {
2226 |       if (this.smsSending) return '发送中...'
2227 |       if (this.smsCooldown > 0) return `${this.smsCooldown} 秒后重发`
2228 |       return '获取验证码'
2229 |     },
2230 |   },
2231 |   onUnload() {
2232 |     this.clearCooldownTimer()
2233 |   },
2234 |   methods: {
2235 |     onPhoneInput(e: UniInputEvent) { this.phone = e.detail.value },
2236 |     onSmsCodeInput(e: UniInputEvent) { this.smsCode = e.detail.value },
2237 |     onNewPwInput(e: UniInputEvent) { this.newPassword = e.detail.value },
2238 |     onConfirmInput(e: UniInputEvent) { this.confirmPassword = e.detail.value },
2239 |     toggleShowPw() { this.showPw = !this.showPw },
2240 |     toggleShowPw2() { this.showPw2 = !this.showPw2 },
2241 |     sanitizePassword(s: string): string {
2242 |       const invisible = /[u200B-u200D\u2060\uFEFF\u00A0\u3000]/g
2243 |       return s.replace(invisible, '').trim()
2244 |     },
2245 |     isValidPhone(s: string): boolean {
2246 |       if (s.length !== 11) return false
2247 |       const re = /^1[3-9]\d{9}$/
2248 |       return re.test(s)
2249 |     },
2250 |     clearCooldownTimer() {
```

```
2251 |   if (this.cooldownTimer !== -1) {
2252 |       clearInterval(this.cooldownTimer)
2253 |       this.cooldownTimer = -1
2254 |   }
2255 | },
2256 | startCooldown(seconds: number) {
2257 |     this.smsCooldown = seconds
2258 |     this.clearCooldownTimer()
2259 |     this.cooldownTimer = setInterval(() => {
2260 |         this.smsCooldown -= 1
2261 |         if (this.smsCooldown <= 0) {
2262 |             this.clearCooldownTimer()
2263 |             this.smsCooldown = 0
2264 |         }
2265 |     }, 1000) as number
2266 | },
2267 | async handleSendSms() {
2268 |     const ph = this.phone.trim()
2269 |     if (!this.isValidPhone(ph)) {
2270 |         uni.showToast({ title: '请输入正确的手机号', icon: 'none' })
2271 |         return
2272 |     }
2273 |     this.smsSending = true
2274 |     try {
2275 |         const res = await authSendSmsCode(buildSendSmsPayload(ph, 'reset')) as any
2276 |         const dev = utsAuthReadDevCode(res)
2277 |         if (dev !== '') {
2278 |             this.smsCode = dev
2279 |             uni.showToast({ title: `验证码已自动填入: ${dev}`, icon: 'none', duration: 2500 })
2280 |         } else {
2281 |             uni.showToast({ title: '验证码已发送', icon: 'success' })
2282 |         }
2283 |         this.startCooldown(60)
2284 |     } catch (error: any) {
2285 |         uni.showToast({
2286 |             title: friendlyAuthError(error, 'sms'),
2287 |             icon: 'none',
2288 |             duration: 2500,
2289 |         })
2290 |     } finally {
2291 |         this.smsSending = false
2292 |     }
2293 | },
2294 | async handleReset() {
2295 |     const ph = this.phone.trim()
2296 |     const code = this.smsCode.trim()
2297 |     const pw = this.sanitizePassword(this.newPassword)
2298 |     const pw2 = this.sanitizePassword(this.confirmPassword)
2299 |     if (!this.isValidPhone(ph)) {
2300 |         uni.showToast({ title: '请输入正确的手机号', icon: 'none' })
```

```
2301 |     return
2302 |   }
2303 |   if (code.length < 4) {
2304 |     uni.showToast({ title: '请填写验证码', icon: 'none' })
2305 |     return
2306 |   }
2307 |   if (pw.length < 6 || pw.length > 32) {
2308 |     uni.showToast({ title: '新密码长度需为 6-32 位', icon: 'none' })
2309 |     return
2310 |   }
2311 |   if (pw !== pw2) {
2312 |     uni.showToast({ title: '两次密码输入不一致', icon: 'none' })
2313 |     return
2314 |   }
2315 |   this.loading = true
2316 |   try {
2317 |     await authResetPassword(buildResetPasswordPayload(ph, code, pw))
2318 |     uni.showToast({ title: '重置成功, 请用新密码登录', icon: 'success' })
2319 |     setTimeout(() => {
2320 |       uni.navigateBack({ fail: () => { uni.redirectTo({ url: '/pages/auth/login' }) } })
2321 |     }, 1000)
2322 |   } catch (error: any) {
2323 |     console.error('[forgot-password] 失败:', error)
2324 |     uni.showToast({
2325 |       title: friendlyAuthError(error, 'reset'),
2326 |       icon: 'none',
2327 |       duration: 2500,
2328 |     })
2329 |   } finally {
2330 |     this.loading = false
2331 |   }
2332 | },
2333 | goBack() {
2334 |   uni.navigateBack({ fail: () => { uni.redirectTo({ url: '/pages/auth/login' }) } })
2335 | },
2336 | },
2337 | }
2338 | </script>
2339 | <style>
2340 | /* 尺寸约定: 与 login / register 对齐; 与 run/* 业务页主流档位一致 */
2341 | .container {
2342 |   flex: 1;
2343 |   justify-content: center;
2344 |   align-items: center;
2345 |   background-color: #0F1426;
2346 |   padding: 24px 24px;
2347 | }
2348 | .box {
2349 |   width: 100%;
2350 |   max-width: 380px;
```

```
2351 | }
2352 | .logo-section {
2353 |   margin-bottom: 20px;
2354 |   align-items: center;
2355 | }
2356 | .logo-text {
2357 |   font-size: 24px;
2358 |   font-weight: bold;
2359 |   color: #F8FAFC;
2360 |   margin-bottom: 6px;
2361 |   letter-spacing: -0.3px;
2362 | }
2363 | .subtitle {
2364 |   font-size: 13px;
2365 |   color: #94A3B8;
2366 | }
2367 | .form-section {
2368 |   width: 100%;
2369 | }
2370 | .input-group {
2371 |   margin-bottom: 14px;
2372 | }
2373 | .label {
2374 |   font-size: 12px;
2375 |   color: #CBD5E1;
2376 |   margin-bottom: 6px;
2377 |   font-weight: bold;
2378 |   letter-spacing: 0.5px;
2379 | }
2380 | .label-row {
2381 |   flex-direction: row;
2382 |   align-items: center;
2383 |   justify-content: space-between;
2384 | }
2385 | .eye {
2386 |   font-size: 12px;
2387 |   color: #A78BFA;
2388 |   margin-bottom: 6px;
2389 |   padding: 2px 6px;
2390 | }
2391 | .eye-hover { opacity: 0.7; }
2392 | .input {
2393 |   width: 100%;
2394 |   height: 44px;
2395 |   padding: 0 14px;
2396 |   border-radius: 12px;
2397 |   font-size: 14px;
2398 |   background-color: #1A2038;
2399 |   color: #F8FAFC;
2400 | }
```



```
2401 | .sms-row {
2402 |   flex-direction: row;
2403 |   align-items: center;
2404 | }
2405 | .sms-input {
2406 |   flex: 1;
2407 |   margin-right: 8px;
2408 | }
2409 | .sms-btn {
2410 |   width: 108px;
2411 |   height: 44px;
2412 |   line-height: 44px;
2413 |   border-radius: 12px;
2414 |   background-color: #A78BFA;
2415 |   color: #F8F AFC;
2416 |   font-size: 13px;
2417 |   font-weight: bold;
2418 |   padding: 0;
2419 | }
2420 | .sms-btn-disabled {
2421 |   background-color: #3E3F6E;
2422 |   opacity: 0.8;
2423 | }
2424 | .primary-btn {
2425 |   width: 100%;
2426 |   height: 44px;
2427 |   line-height: 44px;
2428 |   background-color: #A78BFA;
2429 |   color: #F8F AFC;
2430 |   border-radius: 22px;
2431 |   font-size: 14px;
2432 |   font-weight: bold;
2433 |   margin-top: 10px;
2434 |   margin-bottom: 12px;
2435 | }
2436 | .primary-btn.disabled { opacity: 0.6; }
2437 | .footer-links {
2438 |   margin-top: 12px;
2439 |   align-items: center;
2440 | }
2441 | .link-text {
2442 |   font-size: 13px;
2443 |   color: #A78BFA;
2444 | }
2445 | </style>
2446 | // ===== pages/auth/login.uvue =====
2447 | <template>
2448 |   <view class="login-container">
2449 |     <view class="login-box">
2450 |       <view class="logo-section">
```

```
2451 |     <text class="logo-text">欢迎回来</text>
2452 |     <text class="subtitle">使用账号密码, 或手机号验证码登录</text>
2453 | </view>
2454 | <!-- 登录方式切换 -->
2455 | <view class="tab-row">
2456 |     <view
2457 |         class="tab"
2458 |         :class="{ 'tab-active': mode == 'password' }"
2459 |         hover-class="tab-hover"
2460 |         hover-stay-time="70"
2461 |         @click="switchMode('password')"
2462 |     >
2463 |         <text class="tab-text" :class="{ 'tab-text-active': mode == 'password' }">账号密码</text>
2464 |     </view>
2465 |     <view
2466 |         class="tab"
2467 |         :class="{ 'tab-active': mode == 'sms' }"
2468 |         hover-class="tab-hover"
2469 |         hover-stay-time="70"
2470 |         @click="switchMode('sms')"
2471 |     >
2472 |         <text class="tab-text" :class="{ 'tab-text-active': mode == 'sms' }">短信验证码</text>
2473 |     </view>
2474 | </view>
2475 | <view class="form-section">
2476 |     <!-- 账号密码模式 -->
2477 |     <block v-if="mode == 'password'">
2478 |         <view class="input-group">
2479 |             <text class="label">账号</text>
2480 |             <input
2481 |                 class="input"
2482 |                 type="text"
2483 |                 placeholder="用户名或手机号"
2484 |                 :value="account"
2485 |                 @input="onAccountInput"
2486 |             />
2487 |         </view>
2488 |         <view class="input-group">
2489 |             <text class="label">密码</text>
2490 |             <input
2491 |                 class="input"
2492 |                 type="password"
2493 |                 placeholder="请输入密码"
2494 |                 :value="password"
2495 |                 @input="onPasswordInput"
2496 |             />
2497 |         </view>
2498 |         <button
2499 |             class="primary-btn"
2500 |             @click="handlePasswordLogin"
```

```
2501 |      :disabled="loading"
2502 |    >{{ loading ? '登录中...' : '登录' }}</button>
2503 |  </block>
2504 |  <!-- 短信验证码模式 -->
2505 |  <block v-else>
2506 |    <view class="input-group">
2507 |      <text class="label">手机号</text>
2508 |      <input
2509 |        class="input"
2510 |        type="number"
2511 |        maxlength="11"
2512 |        placeholder="请输入已注册的手机号"
2513 |        :value="phone"
2514 |        @input="onPhoneInput"
2515 |      />
2516 |    </view>
2517 |    <view class="input-group">
2518 |      <text class="label">验证码</text>
2519 |      <view class="sms-row">
2520 |        <input
2521 |          class="input sms-input"
2522 |          type="number"
2523 |          maxlength="6"
2524 |          placeholder="6 位验证码"
2525 |          :value="smsCode"
2526 |          @input="onSmsCodeInput"
2527 |        />
2528 |        <button
2529 |          class="sms-btn"
2530 |          :class="{ 'sms-btn-disabled': smsCooldown > 0 || smsSending }"
2531 |          @click="handleSendSms"
2532 |          :disabled="smsCooldown > 0 || smsSending"
2533 |          >{{ smsBtnLabel }}</button>
2534 |      </view>
2535 |    </view>
2536 |    <button
2537 |      class="primary-btn"
2538 |      @click="handleSmsLogin"
2539 |      :disabled="loading"
2540 |    >{{ loading ? '登录中...' : '登录' }}</button>
2541 |  </block>
2542 |  <!-- 忘记密码 / 去注册 -->
2543 |  <view class="footer-links">
2544 |    <text class="link-text-left" @click="goForgotPassword">忘记密码</text>
2545 |    <text class="link-text-right" @click="goToRegister">还没有账号? 立即注册</text>
2546 |  </view>
2547 |  <!-- 协议提示 -->
2548 |  <view class="agreement-hint">
2549 |    <text class="hint-text">登录即表示同意</text>
2550 |    <text class="hint-link" @click="openUserAgreement">《用户服务协议》</text>
```

```
2551 |     <text class="hint-text">和</text>
2552 |     <text class="hint-link" @click="openPrivacyPolicy">《隐私政策》</text>
2553 | </view>
2554 | <!-- 微信登录（仅小程序端）-->
2555 | <!-- #ifdef MP-WEIXIN -->
2556 | <view class="divider">
2557 |     <view class="divider-line"></view>
2558 |     <text class="divider-text">其它方式</text>
2559 |     <view class="divider-line"></view>
2560 | </view>
2561 | <button
2562 |     class="wechat-btn"
2563 |     @click="handleWechatLogin"
2564 |     :disabled="loading"
2565 | > 微信快捷登录</button>
2566 | <!-- #endif -->
2567 | </view>
2568 | </view>
2569 | </view>
2570 | </template>
2571 | <script lang="uts">
2572 | import {
2573 |   authSignIn,
2574 |   authLoginWithSms,
2575 |   authSendSmsCode,
2576 |   authWechatLogin,
2577 |   buildLoginPayload,
2578 |   buildLoginWithSmsPayload,
2579 |   buildSendSmsPayload,
2580 | } from '@/utils/api'
2581 | import {
2582 |   utsAuthReadToken,
2583 |   utsAuthReadUser,
2584 |   utsAuthReadDevCode,
2585 |   utsWxLoginReadCode,
2586 |   friendlyAuthError,
2587 | } from '@/utils/auth-uts.uts'
2588 | /**
2589 |  * 两种登录方式：
2590 |  *   mode=password: account（用户名或手机号）+ password
2591 |  *   mode=sms      : phone + smsCode
2592 |  * 扁平字段保持和 register.uvue 一致，避免 uni-app-x 原生端对嵌套 data 的怪异 bug。
2593 |  */
2594 | export default {
2595 |   data() {
2596 |     return {
2597 |       mode: 'password' as string, // 'password' | 'sms'
2598 |       account: '' as string,
2599 |       password: '' as string,
2600 |       phone: '' as string,
```

```
2601 |     smsCode: '' as string,
2602 |     loading: false as boolean,
2603 |     // 倒计时 (秒) 。 >0 时按钮置灰 + 显示 "xx秒后重发"
2604 |     smsCooldown: 0 as number,
2605 |     smsSending: false as boolean,
2606 |     cooldownTimer: -1 as number,
2607 |   }
2608 | },
2609 | computed: {
2610 |   smsBtnLabel() : string {
2611 |     if (this.smsSending) return '发送中...'
2612 |     if (this.smsCooldown > 0) return `${this.smsCooldown} 秒后重发`
2613 |     return '获取验证码'
2614 |   },
2615 | },
2616 | onUnload() {
2617 |   this.clearCooldownTimer()
2618 | },
2619 | methods: {
2620 |   onAccountInput(e: UniInputEvent) { this.account = e.detail.value },
2621 |   onPasswordInput(e: UniInputEvent) { this.password = e.detail.value },
2622 |   onPhoneInput(e: UniInputEvent) { this.phone = e.detail.value },
2623 |   onSmsCodeInput(e: UniInputEvent) { this.smsCode = e.detail.value },
2624 |   switchMode(m: string) {
2625 |     if (this.mode !== m) this.mode = m
2626 |   },
2627 |   clearCooldownTimer() {
2628 |     if (this.cooldownTimer !== -1) {
2629 |       clearInterval(this.cooldownTimer)
2630 |       this.cooldownTimer = -1
2631 |     }
2632 |   },
2633 |   startCooldown(seconds: number) {
2634 |     this.smsCooldown = seconds
2635 |     this.clearCooldownTimer()
2636 |     this.cooldownTimer = setInterval(() => {
2637 |       this.smsCooldown -= 1
2638 |       if (this.smsCooldown <= 0) {
2639 |         this.clearCooldownTimer()
2640 |         this.smsCooldown = 0
2641 |       }
2642 |     }, 1000) as number
2643 |   },
2644 |   isValidPhone(s: string): boolean {
2645 |     if (s.length !== 11) return false
2646 |     const re = /^1[3-9]\d{9}$/
2647 |     return re.test(s)
2648 |   },
2649 |   async handleSendSms() {
2650 |     const ph = this.phone.trim()
```

```
2651 |   if (!this.isValidPhone(ph)) {
2652 |       uni.showToast({ title: '请输入正确的手机号', icon: 'none' })
2653 |       return
2654 |   }
2655 |   this.smsSending = true
2656 |   try {
2657 |       const res = await authSendSmsCode(buildSendSmsPayload(ph, 'login')) as any
2658 |       // Mock / 开发模式下后端会把验证码回传，方便联调直接自动填入
2659 |       const dev = utsAuthReadDevCode(res)
2660 |       if (dev !== '') {
2661 |           this.smsCode = dev
2662 |           uni.showToast({ title: `验证码已自动填入：${dev}`, icon: 'none', duration: 2500 })
2663 |       } else {
2664 |           uni.showToast({ title: '验证码已发送', icon: 'success' })
2665 |       }
2666 |       this.startCooldown(60)
2667 |   } catch (error: any) {
2668 |       uni.showToast({
2669 |           title: friendlyAuthError(error, 'sms'),
2670 |           icon: 'none',
2671 |           duration: 2500,
2672 |       })
2673 |   } finally {
2674 |       this.smsSending = false
2675 |   }
2676 | },
2677 | async handlePasswordLogin() {
2678 |     const acct = this.account.trim()
2679 |     const pw = this.password
2680 |     if (acct === '' || pw === '') {
2681 |         uni.showToast({ title: '请填写账号和密码', icon: 'none' })
2682 |         return
2683 |     }
2684 |     this.loading = true
2685 |     try {
2686 |         const raw = await authSignIn(buildLoginPayload(acct, pw)) as any
2687 |         this.afterLoginSuccess(raw)
2688 |     } catch (error: any) {
2689 |         console.error('[login] 密码登录失败:', error)
2690 |         uni.showToast({
2691 |             title: friendlyAuthError(error, 'login'),
2692 |             icon: 'none',
2693 |             duration: 2500,
2694 |         })
2695 |     } finally {
2696 |         this.loading = false
2697 |     }
2698 | },
2699 | async handleSmsLogin() {
2700 |     const ph = this.phone.trim()
```

```
2701 |     const code = this.smsCode.trim()
2702 |     if (!this.isValidPhone(ph)) {
2703 |         uni.showToast({ title: '请输入正确的手机号', icon: 'none' })
2704 |         return
2705 |     }
2706 |     if (code.length < 4) {
2707 |         uni.showToast({ title: '请填写验证码', icon: 'none' })
2708 |         return
2709 |     }
2710 |     this.loading = true
2711 |     try {
2712 |         const raw = await authLoginWithSms(
2713 |             buildLoginWithSmsPayload(ph, code),
2714 |         ) as any
2715 |         this.afterLoginSuccess(raw)
2716 |     } catch (error: any) {
2717 |         console.error('[login] 短信登录失败:', error)
2718 |         uni.showToast({
2719 |             title: friendlyAuthError(error, 'login'),
2720 |             icon: 'none',
2721 |             duration: 2500,
2722 |         })
2723 |     } finally {
2724 |         this.loading = false
2725 |     }
2726 | },
2727 | afterLoginSuccess(raw: any) {
2728 |     uni.setStorageSync('token', utsAuthReadToken(raw))
2729 |     uni.setStorageSync('user', utsAuthReadUser(raw) as any)
2730 |     uni.showToast({ title: '登录成功', icon: 'success' })
2731 |     // profile.uvue 进登录页前 hideTabBar 了, 跳到首页前必须先 show 回来,
2732 |     // 不然 tabBar 会一直消失 (首页 onShow 也会再补一次, 双保险)
2733 |     try { uni.showTabBar({ animation: false }) } catch (_) {}
2734 |     setTimeout(() => {
2735 |         uni.switchTab({ url: '/pages/run/index' })
2736 |     }, 800)
2737 | },
2738 | goForgotPassword() {
2739 |     uni.navigateTo({ url: '/pages/auth/forgot-password' })
2740 | },
2741 | goToRegister() {
2742 |     uni.navigateTo({ url: '/pages/auth/register' })
2743 | },
2744 | openUserAgreement() {
2745 |     uni.navigateTo({ url: '/pages/auth/user-agreement' })
2746 | },
2747 | openPrivacyPolicy() {
2748 |     uni.navigateTo({ url: '/pages/auth/privacy-policy' })
2749 | },
2750 | async handleWechatLogin() {
```

```
2751 | // #ifdef MP-WEIXIN
2752 | this.loading = true
2753 | try {
2754 |   const loginRes = await new Promise((resolve, reject) => {
2755 |     uni.login({
2756 |       provider: 'weixin',
2757 |       success: (res) => { resolve(res) },
2758 |       fail: (err) => { reject(err) },
2759 |     })
2760 |   }) as any
2761 |   const wxCode = utsWxLoginReadCode(loginRes)
2762 |   if (wxCode == '') throw new Error('获取微信登录 code 失败')
2763 |   const rawWx = await authWechatLogin(wxCode) as any
2764 |   this.afterLoginSuccess(rawWx)
2765 | } catch (error: any) {
2766 |   console.error('[login] 微信登录失败:', error)
2767 |   uni.showToast({
2768 |     title: friendlyAuthError(error, 'wechat'),
2769 |     icon: 'none',
2770 |     duration: 2500,
2771 |   })
2772 | } finally {
2773 |   this.loading = false
2774 | }
2775 | // #endif
2776 | // #ifndef MP-WEIXIN
2777 | uni.showToast({ title: '当前平台不支持微信登录', icon: 'none' })
2778 | // #endif
2779 | },
2780 | },
2781 | }
2782 | </script>
2783 | <style>
2784 | /* 尺寸约定（与 register / forgot-password 对齐；与 run/* 业务页主流档位一致）：
2785 | * - 输入框 44 高 / 14 字
2786 | * - 主按钮 44 高 / 22 圆角 / 14 字
2787 | * - logo 24 字，副标题 13 字
2788 | * - 外层 padding 24/24，内层不再加 padding（避免最初版本双层 padding 显得过肥） */
2789 | .login-container {
2790 |   flex: 1;
2791 |   justify-content: center;
2792 |   align-items: center;
2793 |   background-color: #0F1426;
2794 |   padding: 24px 24px;
2795 | }
2796 | .login-box {
2797 |   width: 100%;
2798 |   max-width: 380px;
2799 | }
2800 | .logo-section {
```



```
2801 |   margin-bottom: 20px;
2802 |   align-items: center;
2803 | }
2804 | .logo-text {
2805 |   font-size: 24px;
2806 |   font-weight: bold;
2807 |   color: #F8FAFC;
2808 |   margin-bottom: 6px;
2809 |   letter-spacing: -0.3px;
2810 | }
2811 | .subtitle {
2812 |   font-size: 13px;
2813 |   color: #94A3B8;
2814 | }
2815 | .tab-row {
2816 |   flex-direction: row;
2817 |   background-color: #141B33;
2818 |   border-radius: 22px;
2819 |   padding: 3px;
2820 |   margin-bottom: 20px;
2821 | }
2822 | .tab {
2823 |   flex: 1;
2824 |   height: 36px;
2825 |   justify-content: center;
2826 |   align-items: center;
2827 |   border-radius: 19px;
2828 | }
2829 | .tab-hover {
2830 |   opacity: 0.8;
2831 | }
2832 | .tab-active {
2833 |   background-color: #A78BFA;
2834 | }
2835 | .tab-text {
2836 |   font-size: 13px;
2837 |   color: #94A3B8;
2838 |   font-weight: bold;
2839 | }
2840 | .tab-text-active {
2841 |   color: #F8FAFC;
2842 | }
2843 | .form-section {
2844 |   width: 100%;
2845 | }
2846 | .input-group {
2847 |   margin-bottom: 14px;
2848 | }
2849 | .label {
2850 |   font-size: 12px;
```

```
2851 |   color: #CBD5E1;
2852 |   margin-bottom: 6px;
2853 |   font-weight: bold;
2854 |   letter-spacing: 0.5px;
2855 | }
2856 | .input {
2857 |   width: 100%;
2858 |   height: 44px;
2859 |   padding: 0 14px;
2860 |   border-radius: 12px;
2861 |   font-size: 14px;
2862 |   background-color: #1A2038;
2863 |   color: #F8FAFC;
2864 | }
2865 | .sms-row {
2866 |   flex-direction: row;
2867 |   align-items: center;
2868 | }
2869 | .sms-input {
2870 |   flex: 1;
2871 |   margin-right: 8px;
2872 | }
2873 | .sms-btn {
2874 |   width: 108px;
2875 |   height: 44px;
2876 |   line-height: 44px;
2877 |   border-radius: 12px;
2878 |   background-color: #A78BFA;
2879 |   color: #F8FAFC;
2880 |   font-size: 13px;
2881 |   font-weight: bold;
2882 |   padding: 0;
2883 | }
2884 | .sms-btn-disabled {
2885 |   background-color: #3E3F6E;
2886 |   opacity: 0.8;
2887 | }
2888 | .primary-btn {
2889 |   width: 100%;
2890 |   height: 44px;
2891 |   line-height: 44px;
2892 |   background-color: #A78BFA;
2893 |   color: #F8FAFC;
2894 |   border-radius: 22px;
2895 |   font-size: 14px;
2896 |   font-weight: bold;
2897 |   margin-top: 10px;
2898 | }
2899 | .primary-btn.disabled {
2900 |   opacity: 0.6;
```

```
2901 | }
2902 | .footer-links {
2903 |   flex-direction: row;
2904 |   justify-content: space-between;
2905 |   align-items: center;
2906 |   margin-top: 14px;
2907 | }
2908 | .link-text-left {
2909 |   font-size: 13px;
2910 |   color: #94A3B8;
2911 | }
2912 | .link-text-right {
2913 |   font-size: 13px;
2914 |   color: #A78BFA;
2915 | }
2916 | .agreement-hint {
2917 |   flex-direction: row;
2918 |   flex-wrap: wrap;
2919 |   justify-content: center;
2920 |   margin-top: 16px;
2921 | }
2922 | .hint-text {
2923 |   font-size: 11px;
2924 |   color: #64748B;
2925 |   line-height: 18px;
2926 | }
2927 | .hint-link {
2928 |   font-size: 11px;
2929 |   color: #A78BFA;
2930 |   line-height: 18px;
2931 | }
2932 | .divider {
2933 |   flex-direction: row;
2934 |   align-items: center;
2935 |   margin: 22px 0 14px;
2936 | }
2937 | .divider-line {
2938 |   flex: 1;
2939 |   height: 1px;
2940 |   background-color: #252B45;
2941 | }
2942 | .divider-text {
2943 |   padding: 0 10px;
2944 |   font-size: 12px;
2945 |   color: #94A3B8;
2946 | }
2947 | .wechat-btn {
2948 |   width: 100%;
2949 |   height: 44px;
2950 |   line-height: 44px;
```

行境（uni-app x）前端源代码摘录

第 60 页 / 共 60 页（每页 50 行，不含空行）

```
2951 |   background-color: #07c160;
2952 |   color: #F8FAFC;
2953 |   border-radius: 22px;
2954 |   font-size: 14px;
2955 |   font-weight: bold;
2956 | }
2957 | .wechat-btn:disabled {
2958 |   opacity: 0.6;
2959 | }
2960 | </style>
2961 | // ===== pages/auth/privacy-policy.uvue =====
2962 | <template>
2963 |   <scroll-view class="policy-container" direction="vertical">
2964 |     <view class="content-box">
2965 |       <text class="title">《行境隐私政策》</text>
2966 |       <text class="update-date">更新日期：2025年5月6日</text>
2967 |       <text class="update-date">生效日期：2025年5月6日</text>
2968 |       <text class="intro">「行境」应用（以下简称"本应用"）由行境团队（以下简称"我们"）开发并运营。我们深知个人信息对您的重要性，将严格遵守《中华人民共和国个...
2969 |       <text class="section-title">一、我们收集和使用的个人信息</text>
2970 |       <text class="paragraph">为向您提供服务，我们需要收集以下信息：</text>
2971 |       <text class="sub-title">（一）账号注册与登录</text>
2972 |       <text class="list-item">· 手机号码：用于账号注册、登录验证、找回密码，为必要信息；</text>
2973 |       <text class="list-item">· 用户名和密码：用于创建您的个人账号，为必要信息；</text>
2974 |       <text class="list-item">· 微信授权信息（如使用微信登录）：包括微信openID，用于快捷登录。</text>
2975 |       <text class="sub-title">（二）运动功能</text>
2976 |       <text class="list-item">· 位置信息：用于记录跑步轨迹、计算距离和配速。仅在您使用跑步功能期间获取；</text>
2977 |       <text class="list-item">· 运动与健康数据：包括步数、运动时长、配速等，用于运动数据统计和展示。</text>
2978 |       <text class="sub-title">（三）音乐功能</text>
2979 |       <text class="list-item">· 存储权限：用于本地缓存音乐文件，提升播放体验；</text>
2980 |       <text class="list-item">· 网络权限：用于在线获取音乐资源和BPM信息。</text>
2981 |       <text class="section-title">二、设备权限使用说明</text>
2982 |       <text class="paragraph">为保障本应用各项功能正常运行，我们可能会申请以下设备权限。我们承诺：仅在您主动使用相应功能时才会请求授权，您有权拒绝授权或随时...
2983 |       <view class="permission-card">
2984 |         <text class="permission-name">网络访问</text>
2985 |         <text class="permission-desc">用途：数据同步、账号登录、在线音乐加载等基础功能</text>
2986 |         <text class="permission-required">类型：必要权限</text>
2987 |       </view>
2988 |       <view class="permission-card">
2989 |         <text class="permission-name">精确位置 / GPS</text>
2990 |         <text class="permission-desc">用途：跑步时记录运动轨迹、计算配速和距离</text>
2991 |         <text class="permission-required">类型：使用跑步功能时请求</text>
2992 |       </view>
2993 |       <view class="permission-card">
2994 |         <text class="permission-name">存储 / 文件读写</text>
2995 |         <text class="permission-desc">用途：缓存音乐文件、保存运动数据导出文件</text>
2996 |         <text class="permission-required">类型：使用相关功能时请求</text>
2997 |       </view>
2998 |       <view class="permission-card">
2999 |         <text class="permission-name">读取剪贴板</text>
3000 |         <text class="permission-desc">用途：当您从其他音乐平台复制歌曲链接后，本应用会读取剪贴板内容以识别并导入歌曲信息（如歌名、来源平台等），方便...
```